

Go Restful API

☀ Features

🚀 Core Features

- 🏢 **Domain-Driven Design (DDD)** - (Entities/Value Objects)
- 🏢 **Clean Architecture** - Three-layer architecture (Repository/Service/Handler) with comprehensive dependency injection
- ** 🛠 **Event-Driven Design** **
 - Event-Driven Design : หรือ สถาปัตยกรรมที่ขับเคลื่อนด้วยเหตุการณ์ (Event-Driven Architecture หรือ EDA) คือรูปแบบการออกแบบซอฟต์แวร์ที่
 - ระบบทำงานและสื่อสารกันผ่านการตรวจจับและตอบสนองต่อ เหตุการณ์ (Event) โดยเหตุการณ์หมายถึงการเปลี่ยนแปลงสถานะหรือสิ่งที่เกิดขึ้นในระบบ เช่น การคลิกปุ่ม การสั่งซื้อสินค้า หรือค่าจากเซ็นเซอร์ Event-Driven Design c หรือ สถาปัตยกรรมที่ขับเคลื่อนด้วยเหตุการณ์
- (Event-Driven Architecture หรือ EDA) คือรูปแบบการออกแบบซอฟต์แวร์ที่ระบบทำงานและสื่อสารกันผ่านการตรวจจับและตอบสนองต่อ เหตุการณ์ (Event) โดยเหตุการณ์หมายถึงการเปลี่ยนแปลงสถานะหรือสิ่งที่เกิดขึ้นในระบบ เช่น การคลิกปุ่ม การสั่งซื้อสินค้า หรือค่าจากเซ็นเซอร์
- 🛡 **JWT Authentication** - Complete authentication system with access/refresh tokens and token blacklisting
- 👤 **User Management** - Full CRUD operations with basic role-based protection (admin-only create)
- 📄 **Structured Logging** - Unified logging via `pkg/logger` (built on Go's slog) with trace/request context
- 🚫 **Rate Limiting** - IP-based request throttling with automatic cleanup
- 📊 **Health Monitoring** - Comprehensive health checks with dependency monitoring
- 🌐 **Redis Cache** - Production-ready caching layer with TTL management and object serialization
- 📦 **Message Queue** - Redis-based pub/sub messaging with worker pools and dead letter queue support
- 📁 **Transaction Management** - GORM transaction manager with nested transaction support
- 🛡 **Security** - Multiple security layers including CORS, security headers, and input validation

🔧 Middleware Stack

- **Request Context** - Trace IDs, request IDs, and user context propagation
- **Security Headers** - CSP, HSTS, X-Frame-Options, XSS Protection
- **CORS Handling** - Configurable cross-origin resource sharing
- **Panic Recovery** - Application-level panic handling with graceful error responses
- **Request Logging** - Structured request/response logging with performance metrics
- **Authentication** - JWT middleware with role-based route protection

- **Input Validation** - Comprehensive request validation using `go-playground/validator`

Core Framework & Libraries

- **Web Framework:** `chi/v5` - Lightweight, fast HTTP router with middleware support
- **ORM:** `GORM v1.30.0` - Feature-rich ORM with auto-migration and relations
- **Database Driver:** `gorm.io/driver/postgres` - PostgreSQL driver for GORM
- **Cache:** `github.com/redis/go-redis/v9` - Redis client with pipeline and pub/sub support

Authentication & Security

- **JWT:** `golang-jwt/jwt/v5` - JSON Web Token implementation
- **Password Hashing:** `golang.org/x/crypto/bcrypt` - Secure password hashing
- **Input Validation:** `go-playground/validator/v10` - Struct validation with tags
- **Rate Limiting:** `golang.org/x/time/rate` - Token bucket rate limiting

Configuration & Utilities

- **Configuration:** `spf13/viper` - Configuration management (YAML, ENV, JSON)
- **Logging:** `pkg/logger` - Structured logging (based on Go slog)
- **Testing:** `stretchr/testify` - Testing toolkit with assertions and mocks

Documentation & Development

- **API Documentation:** `swaggo/swag` - Swagger/OpenAPI 3.0 documentation generator
- **HTTP Swagger UI:** `swaggo/http-swagger/v2` - Swagger UI integration

Architecture & Design

Clean Architecture Implementation

- **Handler Layer** - HTTP request handling and response formatting
- **Service Layer** - Business logic and transaction management
- **Repository Layer** - Data access and database operations
- **Dependency Injection** - Interface-based design with comprehensive DI container

Key Design Patterns

- **Repository Pattern** - Abstract data access layer
- **Service Pattern** - Encapsulated business logic
- **Middleware Chain** - Composable request processing
- **Factory Pattern** - Component initialization

- **Observer Pattern** - Configuration watching and hot reload

Security Features

- **JWT Authentication** - Stateless authentication with token blacklisting
- **Role-Based Access Control** - Admin/User role separation
- **Security Headers** - CSP, HSTS, X-Frame-Options, XSS protection
- **Input Validation** - Request validation with custom error messages
- **Rate Limiting** - IP-based request throttling
- **Password Security** - bcrypt hashing with configurable cost

Performance Features

- **Connection Pooling** - Database and Redis connection management
- **Caching Layer** - Redis-based caching with TTL management
- **Structured Logging** - High-performance logging with context
- **Graceful Shutdown** - Zero-downtime deployments
- **Health Checks** - Kubernetes-ready probes



Health & Monitoring

- **Health Endpoints** - Basic, detailed, readiness, and liveness probes
- **Dependency Checks** - Database and Redis connection status via `/health/detailed`
- **System Metrics (Optional)** - Runtime/memory snapshot handler available for wiring
- **Performance Tracking (Optional)** - Monitoring middleware + metrics handler available
- **Kubernetes Ready** - `/ready` and `/live` probes

** 🛠 Event-Driven Design / Event-Driven Architecture (EDA)**

คือรูปแบบการออกแบบซอฟต์แวร์ที่ระบบทำงานและสื่อสารกันผ่าน “เหตุการณ์” (Event) โดยเหตุการณ์หมายถึงการเปลี่ยนแปลงสถานะหรือสิ่งที่เกิดขึ้นในระบบ เช่น การคลิกปุ่ม การสั่งซื้อสินค้า หรือค่าที่อ่านได้จากเซ็นเซอร์ แทนที่ฝ่ายหนึ่งจะเรียกอีกฝ่ายโดยตรง ระบบจะประกาศเหตุการณ์ออกไป แล้วให้ผู้สนใจรับเหตุการณ์นั้นไปประมวลผลต่อเอง

องค์ประกอบหลัก

1. Event Source

แหล่งที่สร้างเหตุการณ์ เช่น ผู้ใช้ เซ็นเซอร์ หรือฐานข้อมูล

2. Event

หน่วยข้อมูลที่บอกว่าจะเกิดขึ้น และโดยปกติจะไม่สามารถแก้ไขได้หลังจากถูกสร้างขึ้นแล้ว (Immutable)

3. Publisher

ผู้ผลิตเหตุการณ์ ทำหน้าที่ส่งเหตุการณ์ออกไปแบบอะซิงโครนัส (Asynchronous) โดยไม่ต้องรอผู้รับ

4. Event Broker / Event Bus

ศูนย์กลางในการรับ คัดกรอง และส่งต่อเหตุการณ์ไปยังผู้บริโภคที่เกี่ยวข้อง

5. Subscriber

ผู้รับเหตุการณ์ คอยฟังและตอบสนองเมื่อมีเหตุการณ์ที่ตนสนใจเกิดขึ้น

หลักการสำคัญ

- **Asynchronous:** ผู้ส่งและผู้รับไม่จำเป็นต้องทำงานพร้อมกัน
- **Decoupling:** แต่ละส่วนไม่ผูกติดกันโดยตรง ทำให้แก้ไขหรือเปลี่ยนระบบได้ง่าย
- **Immutable Event:** เหตุการณ์ที่เกิดขึ้นแล้วถือเป็นข้อเท็จจริง ไม่แก้ไขย้อนหลัง
- **Scalability:** สามารถขยายระบบได้ง่ายขึ้น เพราะแต่ละส่วนทำงานแยกกัน

ข้อดี

- ระบบมีความยืดหยุ่นสูง
- ขยายขนาดได้ง่าย (Scalability)
- แต่ละส่วนทำงานเป็นอิสระต่อกัน
- เหมาะกับระบบ Real-time, Microservices, IoT และ Streaming

ข้อจำกัดและความท้าทาย

- จัดการลำดับของเหตุการณ์ (Event Order) ได้ยาก
- ทดสอบและดีบักระบบกระจายได้ยาก
- อาจเกิดปัญหาเหตุการณ์ซ้ำ (Duplicate Event) หรือเหตุการณ์สูญหาย
- ต้องออกแบบเรื่อง Idempotency, Correlation ID, Observability และ Schema Registry ให้ดี

สรุป

Event-Driven Design เป็นแนวทางที่ช่วยให้ระบบสื่อสารผ่านเหตุการณ์ ลดการผูกติดกันระหว่างบริการ และรองรับการขยายตัวได้ดี แต่ก็แลกมาด้วยความซับซ้อนในการจัดการลำดับเหตุการณ์ การทดสอบ และการติดตาม debugging ในระบบกระจาย ดังนั้นควรใช้เมื่อสถาปัตยกรรมนั้นต้องการความยืดหยุ่นหรือการประมวลผลแบบเรียลไทม์จริง ๆ

อ้างอิง

[1] Medium – The Software Frontier, *Event-Driven Architecture (EDA): A Complete Guide to Real-Time Systems*

[2] GeeksforGeeks – *Event-Driven Architecture System Design*

สรุป Code Review Workflow แบบเข้าใจง่าย

1. Code Assistant — เขียนโค้ดด้วยเครื่องมือช่วย

- ใช้ AI/Code Assistant ช่วยเขียนโค้ดให้เร็วขึ้น
- ตรวจสอบโค้ดเบื้องต้นก่อนส่งขึ้นระบบ เริ่มต้นให้โค้ดพร้อมตรวจ ——

1. Pull Request ส่งโค้ดให้ทีมตรวจ

- เปิด Pull Request พร้อมคำอธิบาย

- แบนรายการเปลี่ยนแปลง, Issue ที่เกี่ยวข้อง
- Reviewer สามารถเข้ามาตรวจได้ทันที เป็นขั้นตอนขอให้ทีมช่วยตรวจ

1. CI Pipeline ตรวจอัตโนมัติ

- ระบบรัน Test / Lint / Build อัตโนมัติ
- ถ้าไม่ผ่าน จะต้องแก้ไขก่อนเข้าสู่ Code Review ป้องกันโค้ดเสียตั้งแต่ต้นทาง

1. Code Review ทีมตรวจโค้ด

- Reviewer ตรวจคุณภาพโค้ด ความถูกต้อง ความเหมาะสม
- อาจมีข้อเสนอแนะหรือให้แก้ไข ให้แน่ใจว่าคุณภาพได้มาตรฐานทีม

1. Code Update แก้ไขตามรีวิว

- ปรับตามคำแนะนำ
- Push โค้ดขึ้นมาใหม่ให้ระบบ rerun รอบแก้ไขเพื่อให้โค้ดสมบูรณ์ขึ้น

1. Deploy — ปลอ่ยสู่ระบบ

- เมื่อรีวิวผ่านและ CI ผ่าน
- โค้ดถูก merge และนำไป deploy ก้าวสู่ production อย่างมั่นใจ

เหมาะกับใคร?

- Developer ทุกระดับ
- ทีมที่ต้องการ workflow ที่ชัดเจน
- ผู้เริ่มต้นที่อยากเรียนรู้ขั้นตอน CI/CD + Code Review
- Tech Lead ที่ต้องการอธิบาย Flow ให้ทีมใหม่เข้าใจเร็ว

สารบัญ

1. บทนำ
2. บทนิยาม
3. ภาพรวมสถาปัตยกรรมระบบ infrastructure
4. เทคโนโลยีและเครื่องมือที่ใช้
5. Queue Management
6. Data Layer และการจัดเก็บข้อมูล
7. Backend Services REST API โดยใช้ Clean Architecture
 - ใช้ (Golang) Golang DDD ใช้ interface ใช้ Modular REST API project structure Redis
 - PostgreSQL queue management system Jwt APIKEY SocketIO
8. Backend Services REST API โดยใช้ Clean Architecture ใช้ (Python) Django and FastAPI
9. Database PostgreSQL InfluxDB
10. Frontend Dashboard ([Tabler.io](https://tabler.io) + Angularjs)
11. Frontend Dashboard ([Tabler.io](https://tabler.io) + ReactJs)

12. AI knime framwork, openclaw framwork
13. Monitoring, Logging และ CI/CD monitoring grafana prometheus
14. Workflow Automation ด้วย [knime](#) [openclaw](#)
15. ความปลอดภัย (Security)
16. การทดสอบระบบ
17. เอกสารประกอบและคู่มือ
18. Workflow การทำงาน
19. Code Review Workflow
20. gitflow workflow action
21. Task Lists และ Checklists
22. ภาคผนวก

หมายเหตุ เนื้อหาในหนังสือ:

เนื้อหาในหนังสือ "ใช้ AI ช่วยเขียน เพื่อทดสอบ AI Model ผู้เขียนเป็นผู้ออกแบบ ใช้ AI ช่วยจัดเรียง ซึ่งมีค่าใช้จ่ายพอสมควร ให้ใช้ฟรีก่อน ต้องการสนับสนุนเพื่อทำเนื้อหาแนวนี้ต่อ สามารถให้การสนับสนุนได้ครับ ตามกำลังศรัทธา 📞

โทรศัพท์ / พร้อมเพย์: **0955088091**

- Code ใช้งานได้จริง

An API dev written in Golang with chi-route and Gorm. Write restful API with fast development and developer friendly.

Architecture

In this project use 3 layer architecture

- Models
- Repository
- Usecase
- Delivery

Features

- CRUD
- Jwt, refresh token saved in redis
- Cached user in redis
- Email verification
- Forget/reset password, send email

Technical

- `chi` : router and middleware
- `viper` : configuration
- `cobra` : CLI features
- `gorm` : orm
- `validator` : data validation
- `jwt` : jwt authentication
- `zap` : logger
- `gomail` : email
- `hermes` : generate email body
- `air` : hot-reload (ใช้ `.\air.cmd` หรือ `go run github.com/air-verse/air@latest` บน Windows 11 ที่ AppLocker block `air.exe`)

Start Application

Generate the Private and Public Keys

- Generate the private and public keys: travistidwell.com/jsencrypt/demo/
- Copy the generated private key and visit this Base64 encoding website to convert it to base64
- Copy the base64 encoded key and add it to the `config/config-local.yml` file as `jwt`
- Similar for public key

Stmp mail config

- Create [mailtrap](#) account
- Create new inboxes
- Update smtp config `config/config-local.yml` file as `smtpEmail`

Run

- `docker-compose up`
- OR go run `./main.go` serve on loca Windows OS
- Swagger: localhost:5000/swagger/
- <http://localhost:5000/swagger/index.html#/>

Hot Reload (air)

```
# Windows 11 ที่ AppLocker block air.exe ใน C:\Users\...\go\bin\air.exe
# ใช้ .air.cmd หรือ go run แทน:
.\air.cmd
```

```
# หรือ
go run github.com/air-verse/air@latest
# หรือ make run
```

```
Email: root@gmail.com
Password: root_password
```

TODO

- Traefik
- Config using .env
- Linter
- Jaeger
- Production docker file version
- Mock database using gomock
- [x] Auth: RefreshToken UC/repo/presenter
- [x] Job: full CRUD + status workflow + service/part/symptom/DTC
- [x] Customer: CRUD + Car management + phone/plate search
- [x] Quotation: CRUD + Approve/Reject + PDF + part/service
- [x] PO: full lifecycle (DRAFT→SENT→CONFIRMED→SHIPPED→RECEIVED)
- [x] Payment: payment + receipt + refund + outstanding balance
- [x] Dashboard: stats/revenue/top-parts/job-status
- [x] Document: file upload/download management
- [x] Email: send + SMTP config + log history
- [x] Batch: batch job CRUD + run + logs
- [x] i18n: multi-language translation
- [x] WOS: web order system
- [x] SQL schema + triggers for all new modules
- [x] Seed data for demo/testing
- [x] Module documentation manual
- [x] All modules compile (`go build ./...`), `go vet ./...` , `go test ./...` pass

Acknowledgements

- github.com/dhax/go-base
- github.com/akmamun/go-fication
- github.com/wpcodevo/golang-fiber-jwt

- github.com/wpcodevo/golang-fiber
- github.com/kienmatu/togo
- github.com/AleksK1NG/Go-Clean-Architecture-REST-API
- github.com/bxcodec/go-clean-arch
- codevoweb.com/golang-and-gorm-user-registration-email-verification/
- codevoweb.com/golang-gorm-postgresql-user-registration-with-refresh-tokens/
- codevoweb.com/how-to-implement-google-oauth2-in-golang/
- codevoweb.com/how-to-upload-single-and-multiple-files-in-golang/
- codevoweb.com/forgot-reset-passwords-in-golang-with-html-email/
- techmaster.vn/posts/34577/kien-truc-sach-voi-golang

Installation

- ตรวจสอบว่า go.mod มี replace directive หรือใช้ local module หรือไม่
 - ถ้า icmongolang เป็น local module ให้ใช้ replace icmongolang => icmongolang
 - หรือถ้าเป็น private repo ให้ตั้ง GOPRIVATE และใช้ access token
 - Perfect! You're setting up an existing Go project (icmongolang). Here's how to properly set it up ar
- ```
```bash
```

Complete Setup Steps for Your icmongolang Project

📁 การจัดการ `go.mod` และ dependencies สำหรับโปรเจกต์ `icmongolang`

📁 Managing `go.mod` and dependencies for `icmongolang` project

> คำแนะนำแบบทีละขั้นตอน (ไทย / อังกฤษ)

> Step-by-step guide (Thai / English)

📁 1. โคลนโปรเจกต์และเข้าไปในโฟลเดอร์

1. Clone and enter the project

```
```bash
```

# ไทย: โคลน repository จาก GitHub และเปลี่ยนไปยังไดเรกทอรีโปรเจกต์

# EN: Clone the repository from GitHub and change into the project directory

```
git clone github.com/kongnakornna/icmongolang.git
```

```
cd icmongolang
```

```
go mod tidy
```

```
go mod download
```

```
go mod verify
```

# ล้างฐานข้อมูลเก่า (ระวังข้อมูล)

# go run cmd/api/main.go migrate:reset (ถ้ามีคำสั่ง)

```
รัน migrate ใหม่
go run main.go migrate
go run ./main.go serve
OR ใช้ go run โดยตรง (ไม่ต้อง build exe)
go run ./cmd/api/main.go serve
go mod tidy
go mod download
go mod verify
go run cmd/api/main.go migrate
go mod vendor
.\air.cmd # หรือ go run github.com/air-verse/air@latest
```

## Auto Run

---

## Windows 11: ใช้ .air.cmd หรือ go run แทน air.exe ที่ถูก AppLocker block

---

.\air.cmd

```
- for windows 10
- .air.toml
 root = "."
 tmp_dir = "tmp"
 env_files = [".env.dev"] # โหลด env โดยอัตโนมัติ

[build]
ใช้ array: [binary, argument1, argument2, ...]
entrypoint = ["../tmp/main.exe", "serve"]
cmd = "go build -o ./tmp/main.exe ./cmd/api"
env = ["GOOS=windows", "GOARCH=amd64"]
clean_on_exit = true

[log]
time = true

[misc]
```

```
clean_on_exit = true
```



## 🚀 โครงสร้างและ Workflow ของโปรเจกต์ icmongolang (Go Backend Clean Architecture)

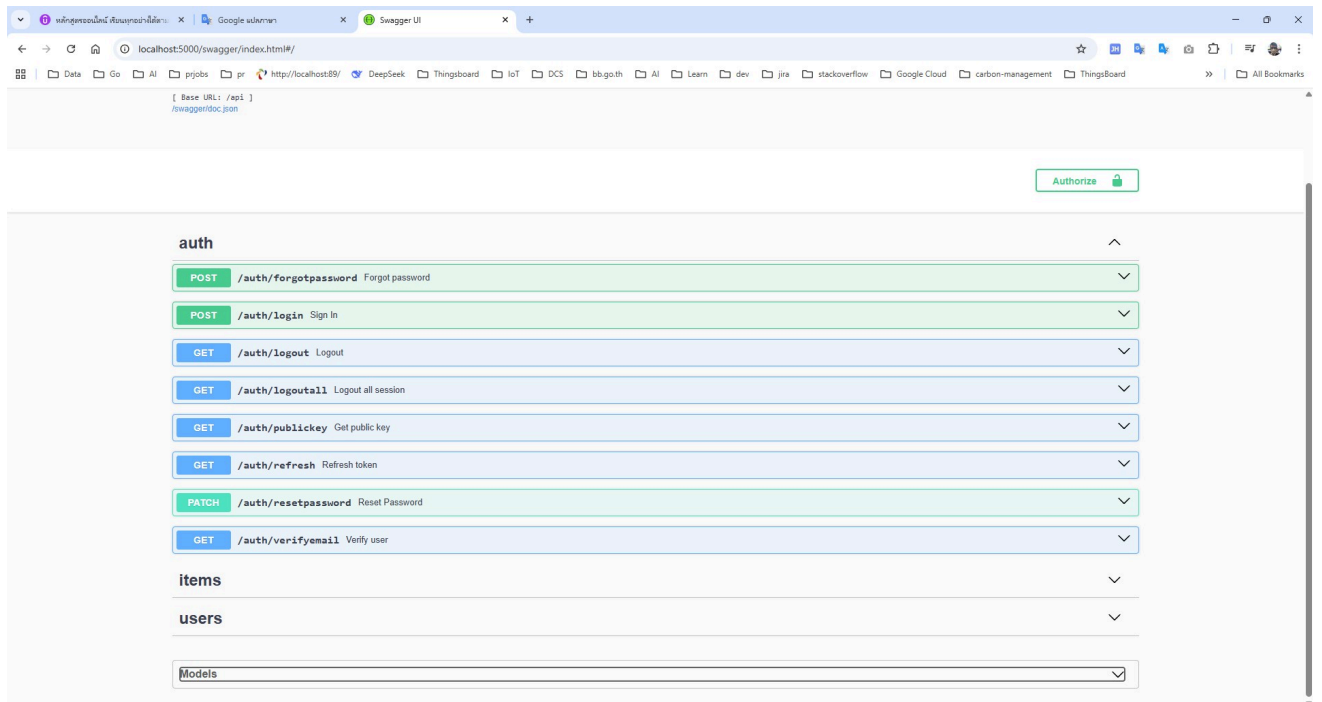
เอกสารนี้ ประกอบด้วย

- โครงสร้างการทำงานแบบละเอียด
- Dataflow Diagram (Flowchart TB สำหรับ [Draw.io](#)) พร้อมคำอธิบาย
- ตัวอย่างโค้ดพร้อมคอมเมนต์ไทย/อังกฤษ ที่รันได้จริง
- กรณศึกษา
- สรุป (ประโยชน์, ข้อควรระวัง, ข้อดี/เสีย, ข้อห้าม, แหล่งอ้างอิง)
- มี Boilerplate พร้อมนำไปใช้

**Boilerplate คือ โค้ดหรือข้อความรูปแบบมาตรฐานที่สามารถนำกลับมาใช้ใหม่ได้หลายครั้งโดยมีการเปลี่ยนแปลงแก้ไขน้อยมากหรือไม่มีเลย**

- มีวัตถุประสงค์หลักเพื่อลดเวลาในการทำงานซ้ำซ้อน เพิ่มมาตรฐานให้กับชิ้นงาน และช่วยให้โครงสร้างไฟล์เริ่มต้นเป็นระเบียบ
- จุดเด่นและประโยชน์ของ Boilerplate:
- ความรวดเร็ว: ไม่ต้องเสียเวลาเขียนโค้ดตั้งต้นใหม่ทุกครั้ง
- มาตรฐาน: สร้างความสม่ำเสมอในโค้ดหรือเอกสาร

- ลดข้อผิดพลาด: เนื่องจากใช้โค้ดที่ผ่านการตรวจสอบมาแล้ว



## 1. โครงสร้างการทำงานของโปรเจกต์ (Architecture Overview)

โปรเจกต์ใช้ **Clean Architecture 3-layer + Delivery**:

Layer	ตำแหน่ง	หน้าที่
<b>Model</b>	internal/models/	Entity (GORM) – User , Session , VerificationToken
<b>Repository</b>	internal/repository/	อ่าน/เขียน DB และ Redis ผ่าน interface
<b>Usecase</b>	internal/usecase/	Business logic: hash, JWT, email queue, validation
<b>Delivery</b>	internal/delivery/rest/	HTTP handlers, middleware, DTO, router
<b>Worker</b>	internal/delivery/worker/	Background job สำหรับส่งอีเมล

### Model คืออะไร?

Models คือโครงสร้างข้อมูล (struct) ที่แทน entity ในฐานข้อมูล หรือข้อมูลที่ใช้ในการสื่อสารระหว่าง layers (DTO) โดยปกติจะสอดคล้องกับตารางใน PostgreSQL และใช้ GORM tags สำหรับ mapping

### มีกี่แบบ?

1. **Entity Model** – สอดคล้องกับตาราง DB โดยตรง (user, session)
2. **DTO (Data Transfer Object)** – ใช้รับ/ส่งข้อมูลระหว่าง API (Request/Response)
3. **Embedded Model** – struct ที่ถูกแทรกใน model อื่น (เช่น BaseModel)

#### 4. Enum-like Model – ใช้ iota สำหรับ status constants

### ใช้อย่างไร / นำไปใช้กรณีไหน

- ใช้ GORM annotations ( `gorm:"column:name;type:..."` ) เพื่อกำหนด schema
- ใช้ JSON tags ( `json:"field_name"` ) สำหรับ serialization
- ใช้ Validator tags ( `validate:"required,email"` ) สำหรับ input validation

### ทำไมต้องใช้

- จัดระเบียบโครงสร้างข้อมูลให้เป็นหนึ่งเดียว
- ช่วยให้ GORM สร้างตารางอัตโนมัติ (AutoMigrate)
- แยก business entity ออกจาก database details

### ประโยชน์ที่ได้รับ

- Type safety ใน Go (ไม่ต้องใช้ `map[string]interface{}`)
- ลด boilerplate code สำหรับ CRUD
- รองรับความสัมพันธ์ระหว่างตาราง (Relationships: BelongsTo, HasMany)

### Boilerplate คือ โค้ดหรือข้อความรูปแบบมาตรฐานที่สามารถนำกลับมาใช้ใหม่ได้หลายครั้งโดยมีการเปลี่ยนแปลงแก้ไขน้อยมากหรือไม่มีเลย

- มีวัตถุประสงค์หลักเพื่อลดเวลาในการทำงานซ้ำซ้อน เพิ่มมาตรฐานให้กับชิ้นงาน และช่วยให้โครงสร้างไฟล์เริ่มต้นเป็นระเบียบ เช่น โครงสร้างพื้นฐานของ HTML หรือการตั้งค่าเริ่มต้นในโปรเจกต์ซอฟต์แวร์ Amazon Web Services
- จุดเด่นและประโยชน์ของ Boilerplate:
- ความรวดเร็ว: ไม่ต้องเสียเวลาเขียนโค้ดตั้งต้นใหม่ทุกครั้ง
- มาตรฐาน: สร้างความสม่ำเสมอในโค้ดหรือเอกสาร
- ลดข้อผิดพลาด: เนื่องจากใช้โค้ดที่ผ่านการตรวจสอบมาแล้ว

### ข้อควรระวัง

- ห้ามเก็บ password plain text (ต้อง hashed)
- ใช้ pointer type สำหรับ nullable fields ( `*time.Time` แทน `time.Time` )
- ระวัง zero values (0, "", false) vs null

### ข้อดี

- ชัดเจน, ตรวจสอบได้ตอน compile
- รองรับ GORM hooks (BeforeCreate, AfterUpdate)

### ข้อเสีย

- ต้องเปลี่ยนแปลง struct เมื่อ schema เปลี่ยน

- อาจมีหลาย struct ที่คล้ายกัน (entity vs response DTO)

## ข้อห้าม

- ห้ามใช้ model สำหรับ business logic (ควรอยู่ใน usecase)
- ห้าม serialize model ที่มี password ไปเป็น JSON

## Repository คืออะไร?

Repository Pattern คือตัวกลาง (abstraction) ระหว่าง business logic (usecase) และแหล่งข้อมูล (database, cache, external API) โดยกำหนด interface สำหรับการเข้าถึงข้อมูล และมี implementation ที่เป็นรูปธรรม (PostgreSQL, Redis) แยกออกจากกัน

## มีกี่แบบ?

1. **Specific Repository** – แต่ละ entity มี interface ของตัวเอง (UserRepository, SessionRepository) – ใช้ในโปรเจกต์นี้
2. **Generic Repository** – interface เดียวที่ใช้กับ entity ใดก็ได้ (ใช้ reflection หรือ interface{})
3. **Transaction Repository** – repository ที่มี method สำหรับ transaction (Begin, Commit, Rollback)
4. **Cached Repository** – decorator ที่เพิ่ม cache layer ให้กับ repository หลัก

## ใช้อย่างไร / นำไปใช้กรณีไหน

- ใช้ interface เพื่อกำหนด method ที่ usecase จะเรียก
- implementation จริงใช้ GORM สำหรับ PostgreSQL และ go-redis สำหรับ Redis
- usecase ไม่รู้ว่าข้อมูลมาจาก DB หรือ Cache
- เหมาะกับระบบที่ต้องเปลี่ยนแหล่งข้อมูลบ่อย หรือต้องการ mock สำหรับ unit test

## ทำไมต้องใช้

- แยก business logic ออกจาก data access code
- ทดสอบ usecase ได้ง่ายโดยใช้ mock repository
- สามารถเปลี่ยนจาก PostgreSQL เป็น MongoDB ได้โดยไม่ต้องแก้ usecase

## ประโยชน์ที่ได้รับ

- ลด dependency coupling
- โค้ดสะอาดขึ้น (Clean Architecture)
- รองรับ caching, logging, monitoring ได้โดยไม่ต้องแก้ usecase

## ข้อควรระวัง

- repository ควรคืนค่าเป็น model ของ domain (internal/models) ไม่ใช่ DTO
- repository ไม่ควรมี business logic (เช่น การตรวจสอบว่า email ซ้ำ ควรอยู่ใน usecase)

- ระวังเรื่อง transaction: ถ้าต้องการ atomic operation ควรส่ง transaction object ( `*gorm.DB` ) เข้าไปใน method

## ข้อดี

- ทดสอบง่าย, เปลี่ยนแหล่งข้อมูลได้, แยกความรับผิดชอบชัดเจน

## ข้อเสีย

- มีโค้ดเพิ่มขึ้น (interface + implementation หลายตัว)
- อาจเพิ่มความซับซ้อนในโปรเจกต์เล็ก

## ข้อห้าม

- ห้ามเรียก repository โดยตรงจาก handler (ต้องผ่าน usecase)
- ห้ามใช้ repository ใน repository อื่น (ควรใช้ service หรือ usecase แทน)
- ห้ามใส่ context ลงใน struct repository (ควรส่งผ่าน method argument)

## Usecase คืออะไร?

Usecase (หรือ Service layer) คือชั้นที่บรรจุ **business logic** ของแอปพลิเคชัน ทำหน้าที่ประสานงานระหว่าง repository ต่างๆ ตรวจสอบกฎทางธุรกิจ และแปลงข้อมูลจากรูปแบบของ repository ให้เป็นรูปแบบที่ delivery (handler) ต้องการ โดย usecase **ไม่รู้** ว่า repository ใช้ PostgreSQL หรือ Redis หรือ external API

## มีกี่แบบ?

1. **Specific Usecase** – แต่ละฟีเจอร์มี usecase ของตัวเอง (AuthUsecase, UserUsecase) – ใช้ในโปรเจกต์นี้
2. **Generic Usecase** – ใช้ interface เดียวกันกับหลาย entity (ไม่ค่อยพบใน Go)
3. **Command/Query Segregation** – แยก Usecase สำหรับการแก้ไขข้อมูล (Command) และการอ่านข้อมูล (Query)
4. **Domain Service** – เมื่อ logic ซับซ้อนและเกี่ยวข้องกับหลาย entity

## ใช้อย่างไร / นำไปใช้กรณีไหน

- ใช้ใน handler: `authUsecase.Login(ctx, email, password)`
- usecase จะเรียก repository method ต่างๆ และอาจเรียกใช้ transaction manager
- คืนค่า business result หรือ error (ไม่คืน HTTP status code)

## ทำไมต้องใช้

- ปกป้อง business logic กระจายอยู่ใน handler หรือ repository
- ทำให้ทดสอบ business logic ได้โดยไม่ต้องมี HTTP request หรือ database จริง (ใช้ mock repository)
- สอดคล้องกับ Clean Architecture

## ประโยชน์ที่ได้รับ

- เปลี่ยน business logic ได้โดยไม่กระทบ delivery (handler) และ repository
- รองรับการ reuse logic (handler เดียวกันใช้ usecase เดียว)
- ง่ายต่อการเพิ่ม logging, tracing, metrics ในชั้นเดียว

## ข้อควรระวัง

- usecase ห้าม import package `net/http` หรือ `gin/chi` เพราะจะทำให้ coupling กับ delivery

## Coupling คืออะไร? (ในบริบทการออกแบบซอฟต์แวร์)

**Coupling (การผูกพัน)** คือระดับที่ โมดูล / คลาส / คอมโพเนนต์ หนึ่งต้องพึ่งพาอีกโมดูลหนึ่ง มากน้อยแค่ไหน

- **High coupling (ผูกพันสูง)** – เปลี่ยนอะไรที่ A แล้ว B พังไปหมด
- **Low coupling (ผูกพันต่ำ)** – แต่ละส่วนเป็นอิสระ เปลี่ยนแปลงได้โดยไม่กระทบกันมาก

## ประเภทของ Coupling (เรียงจากแย่ที่สุดไปดีที่สุด)

ประเภท	คำอธิบาย	ตัวอย่าง
<b>Content coupling</b>	โมดูลเข้าถึงข้อมูลภายในของอีกโมดูลโดยตรง	<code>otherModule.internalVar = 5</code>
<b>Common coupling</b>	ใช้ global variable หรือ shared state ร่วมกัน	<code>var db *sql.DB</code> ทั่วทั้งโปรแกรม
<b>Control coupling</b>	ส่ง flag หรือ parameter เพื่อควบคุมลำดับการทำงานของอีกโมดูล	<code>ProcessData(shouldSave bool)</code>
<b>Stamp coupling</b>	ส่งโครงสร้างข้อมูลที่ใหญ่เกินจำเป็น (ทั้ง struct ทั้งที่ใช้แค่ฟิลด์เดียว)	<code>func Save(user User)</code> แต่ใช้แค่ <code>user.ID</code>
<b>Data coupling</b>	ส่งเฉพาะข้อมูลที่เป็นผ่านพารามิเตอร์	<code>func Save(userID int)</code> ✓
<b>No coupling</b>	ไม่มีการพึ่งพากันเลย (หายาก)	

## ทำไมต้องสนใจ Coupling?

- **Low coupling + High cohesion (การเกาะกลุ่มกันภายใน)** = โค้ดที่บำรุงรักษาง่าย
- ถ้า coupling สูง →
  - แก้ไขที่หนึ่งแล้วกระทบหลายที่
  - ทดสอบยาก (ต้อง mock เยอะ)
  - reuse โมดูลยาก

- เข้าใจระบบยาก

## Coupling กับ Worker ใน Go (เชื่อมกับคำถามก่อนหน้านี้)

ใน pattern **Worker Pool** ถ้าออกแบบไม่ดีจะเกิด coupling สูง เช่น

```
// High coupling: worker รู้จัก database โดยตรง
func worker(jobs <-chan Job) {
 for job := range jobs {
 db.Exec("INSERT ...", job.Data) // coupling กับ DB driver
 }
}
```

วิธีลด coupling:

- ส่ง **interface** ให้ worker แทนคอนกรีต type

```
type Saver interface { Save(data []byte) error }
func worker(jobs <-chan Job, saver Saver) { ... }
```

- ใช้ **dependency injection**
- ใช้ **message queue** (RabbitMQ, Kafka) เป็นตัวกลาง – workers ไม่รู้จักกันและกัน

## ข้อควรปฏิบัติ

- ✓ ชอบ **data coupling / message coupling** (ผ่าน channel หรือ queue)
- ✓ ใช้ **interface** เพื่อลด coupling
- ✓ แยก **business logic** ออกจาก **infrastructure** (DB, HTTP, file)
- ✗ ห้ามใช้ **global state** ร่วมกันระหว่าง workers
- ✗ ห้ามให้ worker เรียก **method** อีก worker โดยตรง (ควรผ่าน channel)

## สรุป

**Coupling** = ระดับการพึ่งพากันระหว่างโมดูล  
**Low coupling** = ดี – เปลี่ยนง่าย, ทดสอบง่าย, reuse ได้  
**High coupling** = ร้าย – โค้ดเปราะบาง, แก้ไขลำบาก

ใน Go การใช้ **channel, interface, dependency injection** ช่วยให้ workers มี coupling ต่ำและยืดหยุ่นมากขึ้น

- usecase ห้าม ส่งออก HTTP status code หรือ JSON
- ควรใช้ interface สำหรับ usecase เพื่อให้ handler มองเห็นแค่ method ที่จำเป็น

## ข้อดี

- แยก business logic ชัดเจน
- ทดสอบ unit ได้ง่าย (ใช้ mock)
- ปรับเปลี่ยน flow ได้โดยไม่แก้ handler

## ข้อเสีย

- เพิ่ม layer ทำให้มีไฟล์มากขึ้น
- มือใหม่อาจเข้าใจยากกว่าควรใส่ logic ตรงไหน (repository หรือ usecase)

## ข้อห้าม

- ห้ามเรียก handler โดยตรงจาก usecase
- ห้ามใช้ `*gorm.DB` ใน usecase (ใช้ repository interface แทน)
- ห้ามใช้ context เพื่อส่งค่าที่ไม่เกี่ยวกับ request (ใช้ argument ปกติ)

## Delivery คืออะไร?

Delivery layer เป็นชั้นที่อยู่ด้านนอกสุดของ Clean Architecture ทำหน้าที่รับ request จากผู้ใช้ (HTTP, gRPC, CLI) แปลงข้อมูล, เรียกใช้ usecase, และส่ง response กลับ โดยไม่มีการประมวลผลทางธุรกิจใดๆ

## มีกี่แบบ?

1. **HTTP/REST handlers** – รับ JSON, เรียก usecase, ส่ง JSON response
2. **Middleware** – ทำงานก่อน/หลัง handlers (authentication, logging, CORS, rate limiting)
3. **WebSocket handlers** – จัดการ real-time connections
4. **gRPC services** – สำหรับ internal microservices
5. **CLI commands** – สำหรับ admin tasks (migrate, seed)

## ใช้อย่างไร / นำไปใช้กรณีไหน

- Handler: แปลง HTTP request → usecase input, usecase output → HTTP response
- Middleware: ตรวจสอบ token, log request, จำกัด rate, เพิ่ม security headers
- DTO: กำหนดโครงสร้าง JSON request/response (แยกจาก entity model)
- Router: จับคู่ path กับ handler และ middleware

## ทำไมต้องใช้

- แยกการรับ/ส่งข้อมูลออกจาก business logic
- เปลี่ยนจาก REST เป็น GraphQL ได้โดยไม่ต้องแก้ usecase
- จัดการ cross-cutting concerns (logging, auth) เป็น centralized

## ประโยชน์ที่ได้รับ

- เปลี่ยนรูปแบบ API (REST → gRPC) โดยไม่กระทบ usecase
- ทดสอบ handler แบบ integration ได้ง่าย
- middleware reuse

## ข้อควรระวัง

- handler ควรสั้น (แค่ binding, validation, call usecase, response)
- อย่าใส่ business logic ใน handler
- DTO ควรแยกจาก entity model เพื่อป้องกันข้อมูลรั่ว (password hash)

## ข้อดี

- แยกชัดเจน, ยืดหยุ่น, middleware จัดการรวม

## ข้อเสีย

- มีไฟล์จำนวนมาก (handler, dto, middleware แต่ละตัว)
- อาจมีการ mapping ซ้ำซ้อน (entity → dto)

## ข้อห้าม

- ห้ามเรียก repository โดยตรงจาก handler
- ห้ามทำ business logic (if-else ที่เกี่ยวกับธุรกิจ) ใน handler
- ห้ามใช้ entity model เป็น request DTO ถ้ามี field ที่ไม่ต้องการให้ client ส่งมา

## Golang: Worker คืออะไร?

Worker ในภาษา Go คือ กระบวนการทำงานเบื้องหลัง (**background process**) ที่ทำงานในรูปแบบ concurrent โดยใช้ **goroutine** และรับงานผ่าน **channel** หรือระบบ queue ต่าง ๆ เพื่อประมวลผลแบบไม่รอ (non-blocking) ช่วยให้โปรแกรมหลักทำงานต่อไปได้โดยไม่ต้องรอผลลัพธ์จากงานหนักหรืองานที่ใช้เวลานาน

## มีกี่แบบ?

แบ่งตามรูปแบบการทำงานและได้ดังนี้

### 1. Single Worker

ใช้ goroutine ตัวเดียวรับงานจาก channel และประมวลผลทีละงาน – เหมาะกับงานที่เรียงลำดับ

### 2. Worker Pool

มี goroutine หลายตัว (จำนวนคงที่) แชร์ channel เดียวกัน รับงานมาแล้วกระจายไปยัง workers – เพิ่ม throughput

### 3. Scheduled / Cron Worker

ทำงานตามเวลาที่กำหนด เช่น ทุกเที่ยงคืน – ใช้ `time.Ticker` หรือ library `cron`

#### 4. CLI Command Worker

ทำงานแบบครั้งเดียวจบ (one-off) สำหรับงานระบบ เช่น

- `go run cmd/migrate/main.go - migrate database`
  - `go run cmd/seed/main.go - seed ข้อมูลเริ่มต้น`
- ทำงานแยกจาก main application ไม่รันตลอดเวลา

#### 5. Message Queue Consumer

ฟัง messages จาก RabbitMQ, Kafka, NATS – workers จะรอและประมวลผลแบบ long-running

## ใช้อย่างไร / นำไปใช้กรณีไหน?

- **API Server** ที่ต้องส่งอีเมล / อัปโหลดไฟล์ – ส่งงานเข้า channel ให้ worker จัดการ async
- **ประมวลผลรูปภาพหรือข้อมูลจำนวนมาก** – ใช้ worker pool แบ่งงานกันทำ
- **งานประจำ (batch jobs)** – สรุปยอดขายทุกเที่ยงคืน, ล้าง cache
- **CLI admin tasks** – `migrate`, `seed`, `backup` ใช้เป็น disposable worker
- **ระบบ Event-driven** – worker คอย consume events จาก Kafka แล้วอัปเดตฐานข้อมูล

ตัวอย่าง Worker Pool ง่าย ๆ ใน Go:

```
func worker(id int, jobs <-chan int, results chan<- int) {
 for job := range jobs {
 results <- job * 2 // simulate work
 }
}

func main() {
 const numWorkers = 5
 jobs := make(chan int, 100)
 results := make(chan int, 100)

 for w := 1; w <= numWorkers; w++ {
 go worker(w, jobs, results)
 }

 // ส่งงาน
 for j := 1; j <= 20; j++ {
 jobs <- j
 }
 close(jobs)
}
```

## ทำไมต้องใช้?

---

- ไม่ **block main goroutine** – โดยเฉพาะใน web server ที่ต้องตอบสนอง client เร็ว
- ใช้ทรัพยากรอย่างคุ้มค่า – Go goroutine มี overhead ต่ำ (เริ่มต้น ~2KB stack)
- แยกความรับผิดชอบ (**separation of concerns**) – โค้ดส่วน worker ไม่ปนกับ business logic
- รองรับการ **scaling** – เพิ่มจำนวน worker ได้ง่ายเมื่อโหลดสูง
- ทำงานขนาน (**parallelism**) – ถ้ามี CPU หลาย core ก็ทำงานพร้อมกันจริง

## ประโยชน์ที่ได้รับ

---

- **Response time** ดีขึ้น – งาน async ไม่ทำให้ client รอานาน
- ใช้ **goroutine** ได้ตรงตามหลัก **Go** – “Do not communicate by sharing memory; instead, share memory by communicating”
- ทนทานต่อ **traffic** สูง – worker pool ควบคุมจำนวน goroutine ไม่ให้หลุดมือ
- โครงสร้างโค้ดสะอาด – แยก worker logic ออกเป็น function / package ได้
- รองรับ **graceful shutdown** – ใช้ `context` สั่งให้ worker หยุดรับงานใหม่และจบงานปัจจุบัน

## ข้อควรระวัง

---

- **Goroutine leak** – ลืมปิด channel หรือไม่รับ signal ให้ worker ออก → memory leak
- **Race condition** – ถ้า workers แชร์โครงสร้างข้อมูลเดียวกัน ต้องใช้ `sync.Mutex` หรือ channel
- **Error handling** – ถ้า worker panic แล้วไม่ recover ทั้งโปรแกรมอาจ crash
- **Buffer overflow** – channel ที่ไม่มี buffer หรือ buffer น้อยเกินไปอาจ deadlock หรือ block
- ไม่กำหนด **timeout** – worker อาจทำงานค้างตลอดไป → ใช้ `context.WithTimeout`
- **Graceful shutdown** ไม่สมบูรณ์ – ทำให้งานค้างหายหรือข้อมูลเสีย

## ข้อดี

---

- น้ำหนักเบา – สร้าง goroutine นับหมื่นได้โดยไม่กิน memory มาก
- ง่ายด้วย **channel** – สื่อสารระหว่าง worker และ main ได้เป็นธรรมชาติ
- **concurrency** สูง – ได้ parallelism โดยไม่ต้องยุ่งยากเรื่อง thread
- เหมาะกับ **I/O-bound** และ **CPU-bound** – ปรับจำนวน worker ตามลักษณะงาน
- มี **pipeline pattern** – ต่อ worker หลายชั้น (fan-out, fan-in) ได้สวยงาม

## ข้อเสีย

---

- ดิบกว่า **synchronous code** – race condition, deadlock หายาก
- ต้องออกแบบ **concurrency** ให้ดี – มิฉะนั้น performance แยกว่า sequential

- **test ยากขึ้น** – ต้องจำลอง channel, time, context
- **overhead เล็กน้อย** – channel และ scheduler มี cost (แต่ต่ำมาก)
- **ควบคุมลำดับงานยาก** – ถ้าต้องการ strict order อาจต้องมี worker เดียว

## ข้อห้าม (สิ่งที่ควรหลีกเลี่ยง)

- **อย่าสร้าง worker แบบไม่จำกัด** – ใช้ worker pool แทน goroutine ระเบิด
- **อย่าใช้ global state ร่วมกันโดยไม่ sync** – ใช้ mutex หรือ channel แทน
- **อย่าละเลย context cancellation** – worker ควรเช็ค `ctx.Done()` เสมอ
- **อย่าปิด channel ขณะมี worker กำลังใช้อยู่** – จะ panic
- **อย่าให้ worker ทำ blocking I/O แบบ infinite** – ควรมี timeout หรือ circuit breaker
- **อย่าใช้ worker สำหรับงานที่ต้อง transaction ACID โดยไม่ careful** – rollback ยาก
- **อย่าเพิกเฉยต่อ worker panic** – ควรมี `recover` และ log

## สรุป

Worker ใน Go เป็น **concurrency pattern** ที่ใช้ **goroutine + channel** เพื่อทำงานเบื้องหลังแบบ async, parallel หรือ scheduled

มีหลายรูปแบบ เช่น single worker, worker pool, cron worker, CLI command worker, message queue consumer

**ประโยชน์หลัก** – ช่วยเพิ่ม throughput, ลด response time, ใช้ทรัพยากรน้อย, รองรับการ scaling

**ข้อควรระวัง** – goroutine leak, race, graceful shutdown, error handling

**ข้อห้าม** – ห้ามสร้าง worker แบบไร้ขีดจำกัด, ห้ามใช้ shared state โดยไม่ป้องกัน, ห้ามลืม context

โดยเฉพาะ **CLI command** (เช่น `migrate`, `seed`) ก็ถูกมองเป็น worker แบบ one-off ที่ใช้สำหรับ admin tasks โดย

เฉพาะ – ทำงานแยกจาก main app ช่วยรักษาความสะอาดของระบบและความปลอดภัย

```
icmongolang/
├── pkg/
│ ├── helpers/
│ │ ├── iot.go # Alarm Logic (สมบูรณ์)
│ │ └── format.go # ฟังก์ชันช่วยเหลือ (time, string, random)
│ └── mqtt/
│ ├── client.go # MQTT client พร้อม GetDataFromTopic
│ └── influxdb/
│ ├── client.go # InfluxDB client
│ └── redis/
│ └── redis_conn.go # Redis client + Cache interface
├── internal/
│ ├── mqtt/
│ │ ├── delivery/http/
│ │ │ ├── handler.go
│ │ │ └── routes.go
│ │ └── presenter/
```

```

| | | └─ presenter.go
| | └─ usecase/
| | └─ usecase.go
| └─ influxdb/
| └─ delivery/http/
| └─ handler.go
| └─ routes.go
| └─ presenter/
| └─ presenter.go
| └─ usecase/
| └─ usecase.go
| └─ alarm/
| └─ delivery/http/
| └─ handler.go
| └─ routes.go
| └─ repository/
| └─ alarm_log_repo.go
| └─ usecase/
| └─ usecase.go
| └─ server/
| └─ handlers.go
| └─ server.go
└─ cmd/api/main.go (สมมติตามเดิม)

```

- NodeJS type script convert to golang GORM entities

## โฟลเดอร์หลัก (Modules)

โปรเจกต์ใช้ **Clean Architecture** 3-layer + Delivery:

### Modules โครงสร้าง

```

internal/modules/
├─ auth/ # Authentication & Authorization
├─ batch/ # Batch/Cron job management
├─ customer/ # Customer & Vehicle management
├─ dashboard/ # Dashboard & Reports
├─ document/ # File upload/download management
├─ email/ # Email service & SMTP config
├─ i18n/ # Multi-language translation
├─ influxdb/ # InfluxDB integration
├─ iot/ # IoT/MQTT v3 device management
├─ items/ # Item CRUD (reference module)
├─ job/ # Job card / repair order
├─ kafka/ # Kafka integration
├─ mqtt/ # Standard MQTT
├─ payment/ # Payment & Receipt

```

- |─ purchaseorder/ # Purchase Order
- |─ quotation/ # Quotation management
- |─ queue/ # In-memory queue
- |─ users/ # User management
- |─ websocket/ # WebSocket server
- |─ wos/ # Web Order System

Layer	ตำแหน่ง	หน้าที่
<b>Model</b>	internal/models/	Entity (GORM) – User , Session , VerificationToken
<b>Repository</b>	internal/repository/	อ่าน/เขียน DB และ Redis ผ่าน interface
<b>Usecase</b>	internal/usecase/	Business logic: hash, JWT, email queue, validation
<b>Delivery</b>	internal/delivery/rest/	HTTP handlers, middleware, DTO, router
<b>Worker</b>	internal/delivery/worker/	Background job สำหรับส่งอีเมล

```

api/
├── cmd/ # Cobra CLI (serve, migrate, initdata, worker)
│ ├── apir/
│ │ └── main.go
│ ├── initdata.go
│ ├── root.go
│ ├── serve.go
│ └── worker.go
├── config/ # Viper config (YAML + env)
│ ├── config.default.yml
│ ├── config.dev.yml
│ └── config.go
├── internal/ # โค้ดส่วนตัว (ไม่ถูก import จากภายนอก)
│ ├── iot/ # shared packages (jwt, redis, email, logger, hash, utils)
│ │ ├── delivery/ # HTTP handlers, middleware, dto, router
│ │ │ └── http/
│ │ │ ├── handler.go
│ │ │ └── routes.go
│ │ ├── worker/
│ │ │ └── worker.go
│ │ ├── helper/ # helper
│ │ │ └── alarm.go
│ │ ├── models/ # GORM entities
│ │ │ ├── alarm.go
│ │ │ ├── common.go
│ │ │ └── device_type.go
│ │ └── presenter/

```

```

| | | └─ presenter.go
| | └─ repository/
| | | └─ alarm_log_repo.go
| | | └─ device_repo.go
| | | └─ schedule_repo.go
| | └─ usecase/ # GORM entities
| | | └─ usecase.go
| | └─ iot.go
| └─ repository/ # interfaces + impl (postgres, redis)
| | └─ pg.go
| | └─ redis.go
| └─ server/
| | └─ handlers.go
| | └─ server.go
| └─ usecase/ # business logic
| | └─ usecase.go
| └─ pg_repository.go
| └─ redis_repository.go
| └─ usecase.go
| └─ pkg/ # shared packages (jwt, redis, email, logger, hash, utils)
| | └─ db/ # DB postgres,redis
| | | └─ postgres/
| | | | └─ db_conn.go
| | | └─ redis/
| | | | └─ redis_conn.go
| | └─ helpers/
| | | └─ iot.go # Alarm Logic (สมบูรณ์)
| | | └─ format.go # ฟังก์ชันช่วยเหลือ (time, string, random)
| | └─ websocket/
| | | └─ websocket.go
| | └─ mqtt/
| | | └─ client.go # MQTT client พร้อม GetDataFromTopic
| | └─ influxdb/
| | | └─ client.go # InfluxDB client
| | └─ httpErrors/
| | | └─ httpErrors.go # httpErrors
| └─ migrations/ # raw SQL (optional)
└─ docker-compose.dev.yml # Postgres + Redis + MailHog
└─ Dockerfile.dev / .air.toml
└─ go.mod

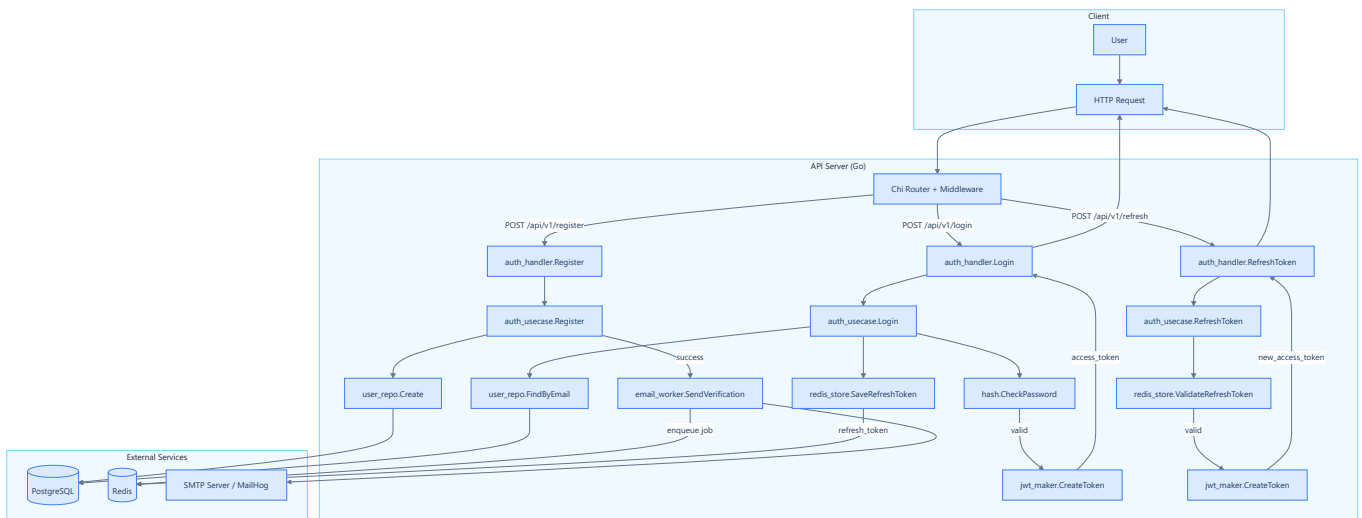
```

# WebSocket server

```
api/
├── cmd/
│ ├── apiser/ # REST API หลัก (มีอยู่แล้ว)
│ ├── websocket/ # *** WebSocket server
│ │ └── main.go
│ ├── initdata.go
│ ├── root.go
│ ├── serve.go
│ └── worker.go
├── internal/
│ ├── websocket/ # **** โค้ดเฉพาะของ WebSocket
│ │ ├── delivery/
│ │ │ └── ws/
│ │ │ ├── hub.go
│ │ │ ├── client.go
│ │ │ └── handler.go # HTTP endpoint สำหรับ upgrade
│ │ ├── usecase/
│ │ │ ├── ws_usecase.go # business logic (save message, auth)
│ │ │ ├── repository/
│ │ │ │ ├── ws_repo.go # interface สำหรับ DB
│ │ │ │ └── models/
│ │ │ │ └── ws_models.go # entity ของ message, session
│ │ └── pkg/ # shared packages (มีอยู่แล้ว)
│ │ ├── websocket/ # **ปรับปรุง** ใช้ร่วมกันได้
│ │ │ ├── hub.go # core Hub logic
│ │ │ ├── client.go
│ │ │ └── message.go # struct ของ message
│ │ └── ...
├── migrations/ # **เพิ่ม** SQL schema สำหรับ websocket
└── 20250619_websocket_tables.sql
```

## 2. Workflow & Dataflow Diagram (Flowchart TB – สำหรับ [Draw.io](https://draw.io))

คัดลอกโค้ด Mermaid ด้านล่างไปวางที่ [draw.io](https://draw.io) (เลือก File → Import from → Mermaid) หรือ [mermaid.live](https://mermaid.live) เพื่อดูภาพจริง



## คำอธิบายแบบละเอียด (Thai / English)

### 2.1 กระบวนการสมัครสมาชิก (Register)

1. **Client** ส่ง `POST /api/v1/register` พร้อม `email`, `password`, `name`
2. **Handler** ตรวจสอบ request → เรียก `auth_usecase.Register`
3. **Usecase**
  - ตรวจสอบอีเมลซ้ำ (เรียก `user_repo.FindByEmail`)
  - Hash รหัสผ่านด้วย `bcrypt`
  - สร้าง `User` object → `user_repo.Create` ลง PostgreSQL
  - สร้าง verification token → เก็บใน Redis (TTL 24h)
  - ส่ง task ไปยัง **email queue** (Go channel หรือ Redis List)
4. **Worker** ดึง task → สร้าง HTML email (Hermes) → ส่งผ่าน SMTP (MailHog หรือ Mailtrap)
5. **Response** `201 Created` กลับไปยัง Client พร้อม message "กรุณายืนยันอีเมล"

### 2.2 กระบวนการล็อกอิน (Login)

1. Client ส่ง `POST /api/v1/login` พร้อม `email`, `password`
2. Usecase ตรวจสอบ credentials
  - `user_repo.FindByEmail`
  - ตรวจสอบ `verified` flag
  - เปรียบเทียบ password กับ hash
3. ถ้าถูกต้อง → สร้าง **access token** (RSA256, อายุ 15 นาที) และ **refresh token** (random 64 chars)
4. เก็บ refresh token ใน Redis: `refresh:{userID}:{tokenID}` → value = userID, TTL 7 วัน
5. Response `200 OK` พร้อม `access_token`, `refresh_token`, `expires_in`

### 2.3 กระบวนการ Refresh Token

1. Client ส่ง `POST /api/v1/refresh` พร้อม `refresh_token` (ใน body หรือ header)

2. Usecase ตรวจสอบ token ใน Redis → ถ้าพบ → สร้าง access token ใหม่
3. Response access token ใหม่ (refresh token อาจถูก renew หรือคงเดิมตาม policy)

## 2.4 กระบวนการ Logout

1. Client ส่ง POST /api/v1/logout พร้อม access token (ใน Authorization header)
2. Middleware ตรวจสอบ JWT → ส่ง user ID ไปยัง Usecase
3. Usecase ลบ refresh token ออกจาก Redis และ optionally ใส่ access token ลง blacklist จนหมดอายุ

## 3. ตัวอย่างโค้ดพร้อมคอมเมนต์ (ไทย/อังกฤษ) และคำอธิบายการใช้งาน

ตัวอย่างนี้เป็น เทมเพลตที่รันได้จริง หลังจากติดตั้ง dependencies และตั้งค่า config แล้ว

### 3.1 internal/usecase/auth\_usecase.go ( Business logic)

```
// Package usecase implements business logic for authentication.
// แพคเกจ usecase มีตรรกะทางธุรกิจสำหรับการรับรองตัวตน
package usecase

import (
 "errors"
 "time"
 "yourmodule/internal/models"
 "yourmodule/internal/repository"
 "yourmodule/internal/pkg/hash"
 "yourmodule/internal/pkg/jwt"
 "yourmodule/internal/pkg/random"
 "yourmodule/internal/pkg/redis"
)

// AuthUsecase defines methods for authentication.
type AuthUsecase interface {
 Register(email, password, name string) (*models.User, error)
 Login(email, password string) (accessToken, refreshToken string, err error)
 RefreshToken(refreshToken string) (newAccessToken string, err error)
 Logout(userID uint, accessToken string) error
}

type authUsecase struct {
 userRepo repository.UserRepository
 redisCache *redis.Client
 jwtMaker jwt.Maker
 emailQueue chan<- models.EmailTask // channel สำหรับส่งงาน async
}
```

```

// NewAuthUsecase creates a new instance with dependency injection.
func NewAuthUsecase(
 ur repository.UserRepository,
 rc *redis.Client,
 jm jwt.Maker,
 eq chan<- models.EmailTask,
) AuthUsecase {
 return &authUsecase{
 userRepo: ur,
 redisCache: rc,
 jwtMaker: jm,
 emailQueue: eq,
 }
}

// Register creates a new user, hashes password, and queues verification email.
// Register สร้าง user ใหม่, hash รหัสผ่าน, และส่งอีเมลยืนยัน (async)
func (a *authUsecase) Register(email, password, name string) (*models.User, error) {
 // ตรวจสอบว่าอีเมลซ้ำหรือไม่ (Check if email already exists)
 existing, _ := a.userRepo.FindByEmail(email)
 if existing != nil {
 return nil, errors.New("email already exists")
 }

 hashedPwd, err := hash.HashPassword(password) // bcrypt
 if err != nil {
 return nil, err
 }

 user := &models.User{
 Email: email,
 PasswordHash: hashedPwd,
 Name: name,
 Verified: false,
 CreatedAt: time.Now().UTC(),
 }

 // บันทึกลง PostgreSQL (Save to DB)
 if err := a.userRepo.Create(user); err != nil {
 return nil, err
 }

 // สร้าง verification token และเก็บใน Redis (Create token, store in Redis)
 verifyToken := random.String(32)
 key := "verify:" + verifyToken
 a.redisCache.Set(key, user.ID, 24*time.Hour)
}

```

```

// ส่ง task ไปยัง email queue (worker จะส่งอีเมลจริง)
a.emailQueue <- models.EmailTask{
 To: email,
 Subject: "Verify your email",
 Template: "verification",
 Data: map[string]interface{}{"Token": verifyToken, "UserID": user.ID},
}

return user, nil
}

// Login checks credentials and returns access + refresh tokens.
// Login ตรวจสอบข้อมูลล็อกอิน และคืน access token + refresh token
func (a *authUsecase) Login(email, password string) (string, string, error) {
 user, err := a.userRepo.FindByEmail(email)
 if err != nil || !user.Verified {
 return "", "", errors.New("invalid credentials or email not verified")
 }

 // เปรียบเทียบรหัสผ่าน (Compare password)
 if err := hash.CheckPasswordHash(password, user.PasswordHash); err != nil {
 return "", "", errors.New("invalid credentials")
 }

 // สร้าง access token (RSA, 15 minutes)
 accessToken, err := a.jwtMaker.CreateToken(user.ID, user.Email, 15*time.Minute)
 if err != nil {
 return "", "", err
 }

 // สร้าง refresh token และเก็บใน Redis (Create refresh token & store in Redis)
 refreshToken := random.String(64)
 refreshKey := "refresh:" + string(rune(user.ID)) + ":" + refreshToken
 a.redisCache.Set(refreshKey, user.ID, 7*24*time.Hour)

 return accessToken, refreshToken, nil
}

```

## 3.2 internal/delivery/rest/handler/auth\_handler.go (HTTP Handler)

```

// Package handler manages HTTP requests for auth endpoints.
// แพคเกจ handler จัดการ HTTP request สำหรับ auth endpoints
package handler

import (

```

```

"net/http"
"yourmodule/internal/delivery/rest/dto"
"yourmodule/internal/usecase"
"github.com/go-playground/validator/v10"
)

type AuthHandler struct {
 authUC usecase.AuthUsecase
 validate *validator.Validate
}

func NewAuthHandler(au usecase.AuthUsecase) *AuthHandler {
 return &AuthHandler{authUC: au, validate: validator.New()}
}

// Register handles POST /register
// @Summary Register a new user
// @Accept json
// @Param request body dto.RegisterRequest true "Registration info"
// @Success 201 {object} dto.UserResponse
// @Failure 400 {object} dto.ErrorResponse
func (h *AuthHandler) Register(w http.ResponseWriter, r *http.Request) {
 var req dto.RegisterRequest
 if err := dto.DecodeJSON(r, &req); err != nil {
 dto.RespondError(w, http.StatusBadRequest, "invalid request body")
 return
 }
 if err := h.validate.Struct(req); err != nil {
 dto.RespondError(w, http.StatusBadRequest, err.Error())
 return
 }

 user, err := h.authUC.Register(req.Email, req.Password, req.Name)
 if err != nil {
 dto.RespondError(w, http.StatusConflict, err.Error())
 return
 }

 dto.RespondJSON(w, http.StatusCreated, dto.ToUserResponse(user))
}

// Login handles POST /login
func (h *AuthHandler) Login(w http.ResponseWriter, r *http.Request) {
 var req dto.LoginRequest
 if err := dto.DecodeJSON(r, &req); err != nil {
 dto.RespondError(w, http.StatusBadRequest, "invalid request")
 return
 }

```

```

 }

 accessToken, refreshToken, err := h.authUC.Login(req.Email, req.Password)
 if err != nil {
 dto.RespondError(w, http.StatusUnauthorized, err.Error())
 return
 }

 dto.RespondJSON(w, http.StatusOK, dto.TokenResponse{
 AccessToken: accessToken,
 RefreshToken: refreshToken,
 TokenType: "Bearer",
 ExpiresIn: 900, // 15 minutes
 })
}

```

### 3.3 internal/pkg/jwt/rsa\_maker.go (JWT RSA sign/verify)

```

// Package jwt provides RSA-based JWT creation and verification.
// แพ็คเกจ jwt ให้การสร้างและตรวจสอบ JWT ด้วย RSA
package jwt

import (
 "crypto/rsa"
 "errors"
 "time"
 "github.com/golang-jwt/jwt/v5"
)

type RSAMaker struct {
 privateKey *rsa.PrivateKey
 publicKey *rsa.PublicKey
}

func NewRSAMaker(privatePEM, publicPEM []byte) (*RSAMaker, error) {
 privateKey, err := jwt.ParseRSAPrivateKeyFromPEM(privatePEM)
 if err != nil {
 return nil, err
 }
 publicKey, err := jwt.ParseRSAPublicKeyFromPEM(publicPEM)
 if err != nil {
 return nil, err
 }
 return &RSAMaker{privateKey: privateKey, publicKey: publicKey}, nil
}

```

```
// CreateToken สร้าง JWT signed ด้วย private key (RSA256)
func (m *RSAMaker) CreateToken(userID uint, email string, duration time.Duration) (string, error) {
 claims := jwt.MapClaims{
 "sub": userID,
 "email": email,
 "exp": time.Now().Add(duration).Unix(),
 "iat": time.Now().Unix(),
 }
 token := jwt.NewWithClaims(jwt.SigningMethodRS256, claims)
 return token.SignedString(m.privateKey)
}

// VerifyToken ตรวจสอบ JWT ด้วย public key
func (m *RSAMaker) VerifyToken(tokenString string) (*Payload, error) {
 token, err := jwt.Parse(tokenString, func(t *jwt.Token) (interface{}, error) {
 if _, ok := t.Method.(*jwt.SigningMethodRSA); !ok {
 return nil, errors.New("unexpected signing method")
 }
 return m.publicKey, nil
 })
 if err != nil {
 return nil, err
 }
 claims, ok := token.Claims.(jwt.MapClaims)
 if !ok || !token.Valid {
 return nil, errors.New("invalid token")
 }
 return &Payload{
 UserID: uint(claims["sub"].(float64)),
 Email: claims["email"].(string),
 }, nil
}
```

### 3.4 Docker Compose สำหรับ Development ( docker-compose.dev.yml )

```
version: '3.8'
services:
 postgres:
 image: postgres:15-alpine
 environment:
 POSTGRES_USER: appuser
 POSTGRES_PASSWORD: secret
 POSTGRES_DB: icmongo
 ports:
 - "5432:5432"
 volumes:
```

```

- pgdata:/var/lib/postgresql/data

redis:
 image: redis:7-alpine
 ports:
 - "6379:6379"

mailhog:
 image: mailhog/mailhog
 ports:
 - "1025:1025" # SMTP
 - "8025:8025" # Web UI

volumes:
 pgdata:

```

### 3.5 วิธีรันโปรเจกต์ (Run Step-by-Step)

```

1. สร้าง RSA key pair สำหรับ JWT
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem

2. ตั้ง environment variables (หรือใช้ .env)
export DB_URL="postgres://appuser:secret@localhost:5432/icmongo?sslmode=disable"
export REDIS_ADDR="localhost:6379"
export JWT_PRIVATE_KEY_PATH="private.pem"
export JWT_PUBLIC_KEY_PATH="public.pem"
export SMTP_HOST="localhost"
export SMTP_PORT="1025"

3. รัน infrastructure services
docker-compose -f docker-compose.dev.yml up -d

4. รัน migration (สมมติใช้ GORM AutoMigrate)
go run cmd/api/main.go migrate

5. รัน background worker (อีกเทอร์มินัล)
go run cmd/api/main.go worker

6. รัน HTTP server (หรือใช้ air เพื่อ hot-reload)
go run ./main.go serve
หรือ
air -c .air.toml

7. ทดสอบ API
curl -X POST http://localhost:5000/api/v1/register \

```

```
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","password":"P@ssw0rd","name":"Test User"}'

curl -X POST http://localhost:5000/api/v1/login \
-H "Content-Type: application/json" \
-d '{"email":"test@example.com","password":"P@ssw0rd"}'
```

## 4. กรณีศึกษา (Case Study)

บริษัท E-commerce ขนาดกลาง ต้องการระบบสมาชิกที่รวดเร็วและปลอดภัย

- ปัญหาเดิม: ระบบ legacy ใช้ session บนไฟล์, ไม่รองรับ mobile, ส่งอีเมลแบบ sync ทำให้ response ช้า
- Solution ด้วยโครงสร้างนี้:
  - **JWT + Redis** ทำให้ Stateless authentication, mobile ใช้งานง่าย
  - **Background email queue** ช่วยให้ register/login response เร็ว (< 50ms)
  - **RSA JWT** ช่วยให้ฝั่ง mobile ตรวจสอบ token ได้เอง (ถ้ามี public key)
  - **Rate limiter** (Redis sliding window) ป้องกัน brute force
- ผลลัพธ์:
  - รองรับ user 200,000 คน, 并发 5,000 req/s
  - Developer onboarding ลดลง 60% เพราะโครงสร้างมาตรฐาน
  - สามารถ deploy แบบ containerized บน Kubernetes ได้ง่าย

## 5. สรุป

### ✓ ประโยชน์ที่ได้รับ

- **Clean Architecture** – แยกหน้าที่, ทดสอบง่าย (mock repository)
- **Stateless JWT + Redis** – scale แนวนอนได้, ไม่ต้อง sticky session
- **Async email** – API response เร็ว, ไม่ติด I/O
- **Hot-reload (Air)** – เพิ่ม productivity ใน development
- **Docker Compose** – dev environment 一致, ลด “มันทำงานบนเครื่องฉัน”

### ⚠ ข้อควรระวัง

- **RSA private key** ต้องเก็บให้ปลอดภัย (ใช้ Vault หรือ K8s secret)
- **Refresh token rotation** – ต้อง implement blacklist และป้องกัน replay
- **Redis เป็น Single Point of Failure** – สำหรับ production ควรใช้ Redis Sentinel หรือ Cluster
- **GORM AutoMigrate** ไม่เหมาะกับ production (ควรใช้ `golang-migrate` หรือ `goose`)
- **Email queue** ต้องมี dead-letter และ monitoring

## 👍 ข้อดี

- Performance สูง (Go + Chi + GORM)
- โครงสร้างเป็นมาตรฐาน ทำให้ทีมงานเข้าใจง่าย
- สามารถเปลี่ยน database หรือ cache ได้โดยไม่ต้องแก้ usecase
- รองรับ unit test และ integration test ดี (เพราะใช้ interface)

## 🗨 ข้อเสีย

- Boilerplate code ค่อนข้างมาก (repository, usecase, dto)
- มือใหม่ที่ยังไม่เข้าใจ Clean Architecture อาจสับสนกับการแมป layer
- การใช้ Go channel เป็น email queue เหมาะกับ monolith; ถ้าต้อง distributed ควรใช้ RabbitMQ / Kafka

## 🚫 ข้อห้าม (ต้องห้าม)

- ห้ามเก็บ **plain-text password** – ต้อง bcrypt หรือ argon2
- ห้ามตั้ง **JWT expiry** เกิน 1 วัน (ควร 15-30 นาที)
- ห้ามส่ง **access token** ผ่าน **URL parameter** – ใช้ `Authorization: Bearer` เท่านั้น
- ห้าม **commit private key** ลง **Git** – ใส่ใน `.gitignore` เสมอ
- ห้ามใช้ **GORM AutoMigrate** ใน **production** โดยไม่ตรวจสอบ **schema change**

## 📚 แหล่งอ้างอิง / ที่มา

- [Clean Architecture in Go \(Uncle Bob\)](#)
- [JWT with RSA – RFC 7519](#)
- [GORM, Chi router](#)
- [Uber Go Style Guide](#)
- [Air – live reload for Go](#)
- [Codevoweb – Go + GORM tutorials](#)
- [Techmaster – Clean Architecture in Go](#)
- Template ต้นแบบจาก [github.com/dhax/go-base](https://github.com/dhax/go-base) และ [github.com/AleksK1NG/Go-Clean-Architecture-REST-API](https://github.com/AleksK1NG/Go-Clean-Architecture-REST-API)

หมายเหตุ: ตัวอย่างโค้ดทั้งหมดอยู่ใน namespace `yourmodule` – คุณควรเปลี่ยนเป็น `icmongolang` ตามชื่อโปรเจกต์จริง และต้องติดตั้ง dependencies ด้วย `go mod tidy` ก่อนรัน

# คำอธิบายการทำงานตามโครงสร้าง icmongolang

## 1. โครงสร้างนี้คืออะไร?

โครงสร้าง icmongolang คือ เทมเพลตสำหรับพัฒนา REST API ด้วยภาษา Go ที่ใช้ Clean Architecture (หรือเรียกอีกแบบว่า Layered Architecture) โดยแบ่งชั้นหน้าที่ชัดเจน 3-4 ชั้น ได้แก่:

- **Model Layer** ( internal/models/ ) – กำหนดโครงสร้างข้อมูล (entity) ที่สอดคล้องกับฐานข้อมูล
- **Repository Layer** ( internal/repository/ ) – ติดต่อฐานข้อมูลและ Redis โดยใช้ interface
- **Usecase Layer** ( internal/usecase/ ) – จัดการ business logic (การ hash password, สร้าง JWT, ส่งอีเมล async, ฯลฯ)
- **Delivery Layer** ( internal/delivery/ ) – รับผิชอบ HTTP handler, middleware, DTO, router (รวมถึง background worker สำหรับอีเมล)

นอกจากนี้ยังมี pkg/ สำหรับ shared packages (JWT, Redis client, logger, validator, email sender) และ cmd/ สำหรับ CLI commands (Cobra) เช่น การรัน server, migrate, init data, worker

## 2. มีกี่แบบ (รูปแบบของสถาปัตยกรรม)

โครงสร้างนี้มี 3 รูปแบบหลัก ที่ซ้อนกันอยู่:

แบบ	คำอธิบาย	ปรากฏในโครงสร้างนี้หรือไม่
3-Layer Architecture	Presentation (Delivery) – Business () – Data (Repository)	✓ ใช่ – เป็นพื้นฐาน
Clean Architecture	วงในสุดคือ Entity (Model), วงถัดมา , วงนอก Delivery และ Repository (dependency inversion)	✓ ใช่ – ใช้ interface ทำให้ dependency ซี่เข้าใน
Modular Monolith	โครงสร้างภายในแบ่งตาม module (auth, user, item) แต่ยัง compile เป็น binary เดียว	✓ ใช่ – มีโฟลเดอร์ auth , users , items อยู่ใน internal/

นอกจากนี้ยังมี รูปแบบการทำงานของ CLI แบบ Command Pattern (Cobra) และ Background Worker แบบ Queue (channel หรือ Redis Stream)

### 3. ใช้อย่างไร? (วิธีใช้งานแต่ละ layer)

#### 3.1 การทำงานของแต่ละ layer (จาก request ไป response)

```
Client Request → Router (Chi) → Middleware → Handler (Delivery) → Repository → DB/Redis
 ← Response ← ← (return) ←
```

รายละเอียด:

- **Router** ( `router.go` ) กำหนดเส้นทางและ middleware (CORS, Logger, RateLimit, Auth)
- **Handler** ( `*_handler.go` ) รับ request → แปลงเป็น DTO → เรียก → แปลง response → ส่ง JSON กลับ
- **\*\*\*\*** ( `*_.go` ) ทำงาน business logic โดยไม่สนใจว่า data มาจาก DB หรือ cache หรือ external API
- **Repository** ( `*_repo.go` ) เป็น interface ที่ เรียก; implementation จะติดต่อ PostgreSQL (GORM) หรือ Redis (go-redis)
- **Model** ( `models/*.go` ) เป็น struct ที่ GORM ใช้ map กับตาราง

#### 3.2 Dependency Injection (DI)

ต้องการ Repository → สร้าง Repository ก่อนแล้วส่งเข้า NewUsecase

Handler ต้องการ Usecase → สร้าง Usecase แล้วส่งเข้า NewHandler

Router ต้องการ Handler → สร้าง Handler แล้ว register route

ตัวอย่างใน `main.go` หรือ `cmd/serve.go` :

```
userRepo := repository.NewUserRepository(db)
redisClient := redis.NewClient(cfg.Redis)
jwtMaker := jwt.NewRSAMaker(privateKey, publicKey)
authUC := usecase.NewAuthUsecase(userRepo, redisClient, jwtMaker, emailQueue)
authHandler := handler.NewAuthHandler(authUC)
router.RegisterAuthRoutes(authHandler)
```

### 4. นำไปใช้ในกรณีใด?

กรณี	เหมาะสมหรือไม่
ระบบ authentication / user management	✅ เหมาะมาก – มี JWT, refresh token, email verify, forget password ครบ
ระบบ e-commerce หรือ membership	✅ เหมาะ – มี RBAC (role), caching, background worker

กรณี	เหมาะสมหรือไม่
ระบบที่ต้องรองรับการเปลี่ยนแปลง database	✅ เหมาะ – เพราะ repository เป็น interface เปลี่ยนจาก PostgreSQL เป็น MySQL ได้โดยไม่แก้ usecase
API ที่ต้องการ performance สูง	✅ เหมาะ – Go + Chi + GORM + Redis ให้ throughput ดี
ระบบ monolith ขนาดกลางถึงใหญ่	✅ เหมาะ – แยก module ชัดเจน (auth, users, items)
ระบบที่ต้องการ distributed / microservices	⚠️ ต้องปรับ – เปลี่ยน channel queue เป็น message broker (RabbitMQ, Kafka) และใช้ API gateway
ระบบ(CRUD ธรรมดา)	❌ เกินความจำเป็น – อาจใช้แค่ Gin + GORM ก็พอ

## 5. ทำไมต้องใช้โครงสร้างนี้? (เหตุผล)

1. แยกความรับผิดชอบ (Separation of Concerns) – แต่ละ layer ทำหน้าที่ของตัวเอง ไม่ยุ่งกัน
2. ทดสอบง่าย (Testability) – สามารถ mock repository หรือ usecase ได้โดยไม่ต้องต่อ database จริง
3. เปลี่ยนเทคโนโลยีได้โดยไม่กระทบ logic – เช่น เปลี่ยนจาก GORM เป็น sqlx ก็แค่แก้ repository, usecase ไม่ต้องเปลี่ยน
4. ลด coupling – ใช้ interface ทำให้ layer ภายในไม่รู้จัก layer นอก
5. รองรับการขยาย (Scalability of code) – เพิ่ม feature ใหม่โดยไม่พัง feature เดิม
6. ทำงานเป็นทีมได้ง่าย – นักพัฒนาสามารถทำ handler, usecase, repository พร้อมกันโดยไม่ชนกัน

## 6. ประโยชน์ที่ได้รับ

### ✅ ด้านการพัฒนา

- Hot-reload (Air) – แก้โค้ดแล้วเซฟ แอป restart อัตโนมัติ
- CLI commands – migrate, seed data, start worker แยกออกจาก server
- Swagger docs – สร้างอัตโนมัติจาก annotation

### ✅ ด้านความปลอดภัย

- JWT with RSA – ไม่ต้องแชร์ secret key ระหว่าง services
- Refresh token ใน Redis – สามารถ revoke ได้
- Bcrypt hash password – ป้องกัน credential leak
- Rate limiter – ป้องกัน brute force

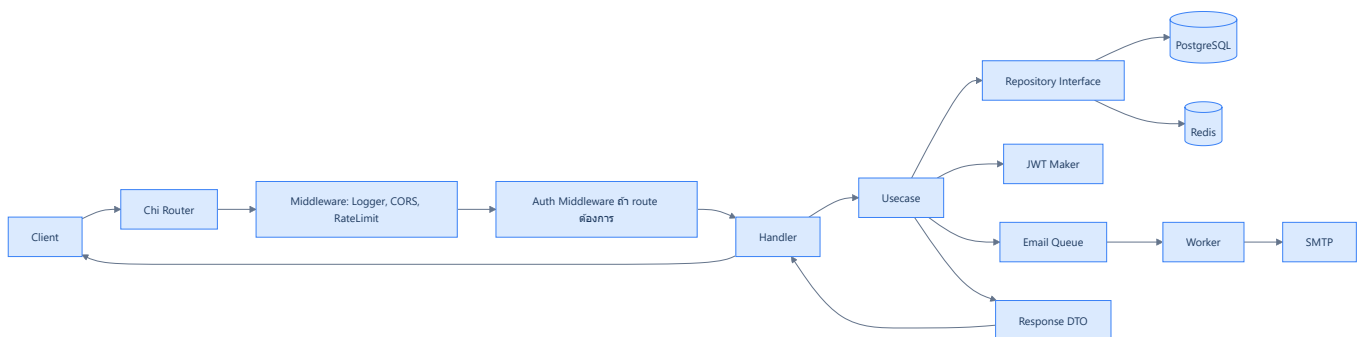
## ✓ ด้าน performance

- **Redis cache** – ลด load database
- **Background email queue** – API response ไม่ติด I/O
- **GORM preloading & connection pool** – จัดการ query ได้ดี

## ✓ ด้าน maintenance

- โครงสร้างเป็นมาตรฐาน – developer ใหม่เข้าใจเร็ว
- เปลี่ยน database หรือ cache ได้โดยไม่แก้ business logic
- ใช้ dependency injection – เปลี่ยน implementation ได้ง่าย

## 7. โครงสร้างการทำงาน (Flow ของ request)



## 8. ออกแบบ Workflow (ยกตัวอย่าง 2 workflow หลัก)

### 8.1 Workflow การสมัครสมาชิก (Register)

1. **Client** → POST /api/v1/register (email, password, name)
2. **Handler** → validate DTO → เรียก authUsecase.Register
3. **Usecase**
  - userRepo.FindByEmail ตรวจสอบอีเมลซ้ำ
  - hash.HashPassword
  - userRepo.Create ลง PostgreSQL
  - สร้าง verification token → redis.Set (TTL 24h)
  - ส่ง task ไปยัง emailQueue (channel)
4. **Worker** (รันใน goroutine อื่น)
  - ดึง task จาก queue
  - สร้าง HTML email ด้วย Hermes
  - ส่งผ่าน SMTP (MailHog หรือ Mailtrap)
5. **Handler** → response 201 Created ทันที (ไม่รออีเมล)

## 8.2 Workflow การล็อกอิน (Login)

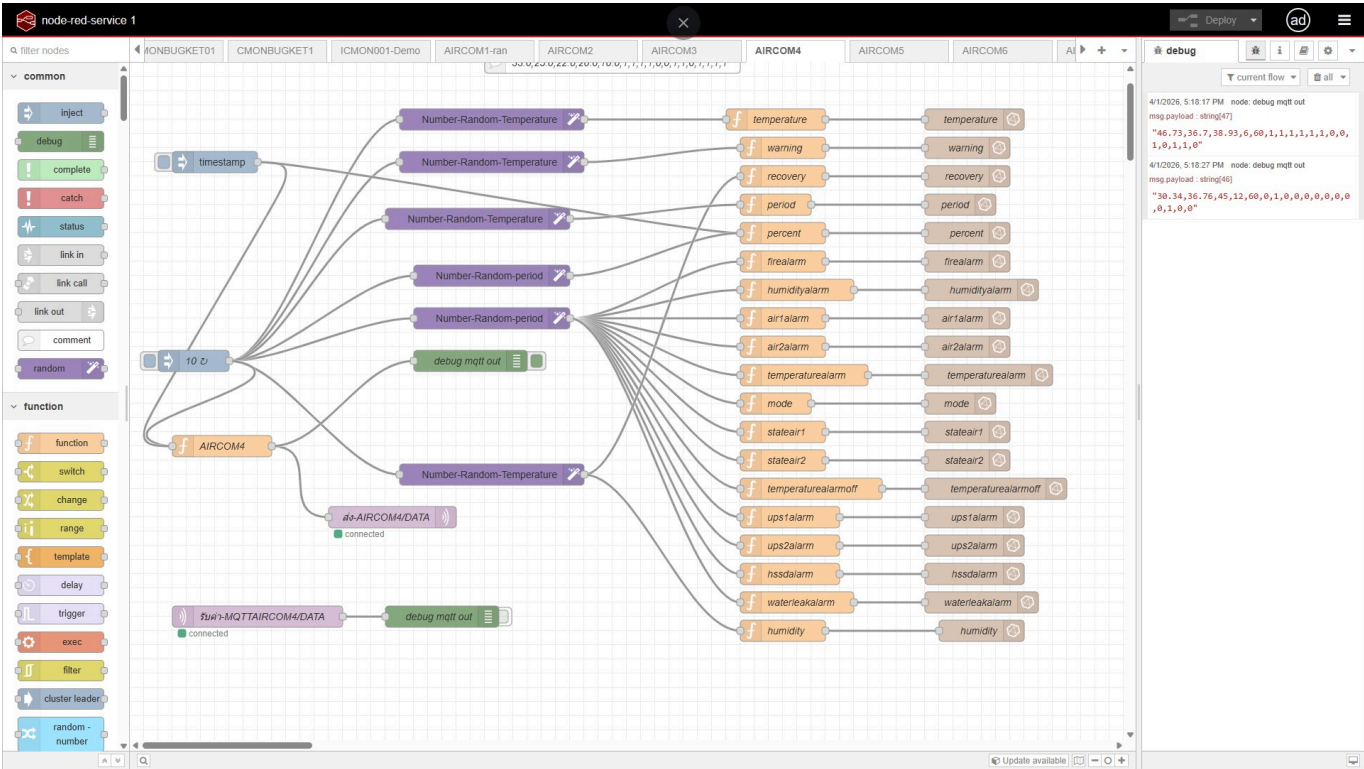
1. **Client** → POST /api/v1/login (email, password)
2. **Usecase**
  - `userRepo.FindByEmail`
  - ตรวจสอบ `user.Verified` และ password
  - `jwtMaker.CreateToken` สร้าง access token (RSA, 15m)
  - สร้าง refresh token (random string)
  - `redis.Set` เก็บ refresh token (key: `refresh:{userID}:{tokenID}` , TTL 7d)
3. **Handler** → response 200 OK พร้อม `access_token` , `refresh_token`
4. **Client** เก็บ token (localStorage / secure cookie)
5. เมื่อ access token หมดอายุ → Client ส่ง POST /api/v1/refresh พร้อม refresh token
6. **Usecase** → ตรวจสอบ refresh token ใน Redis → สร้าง access token ใหม่

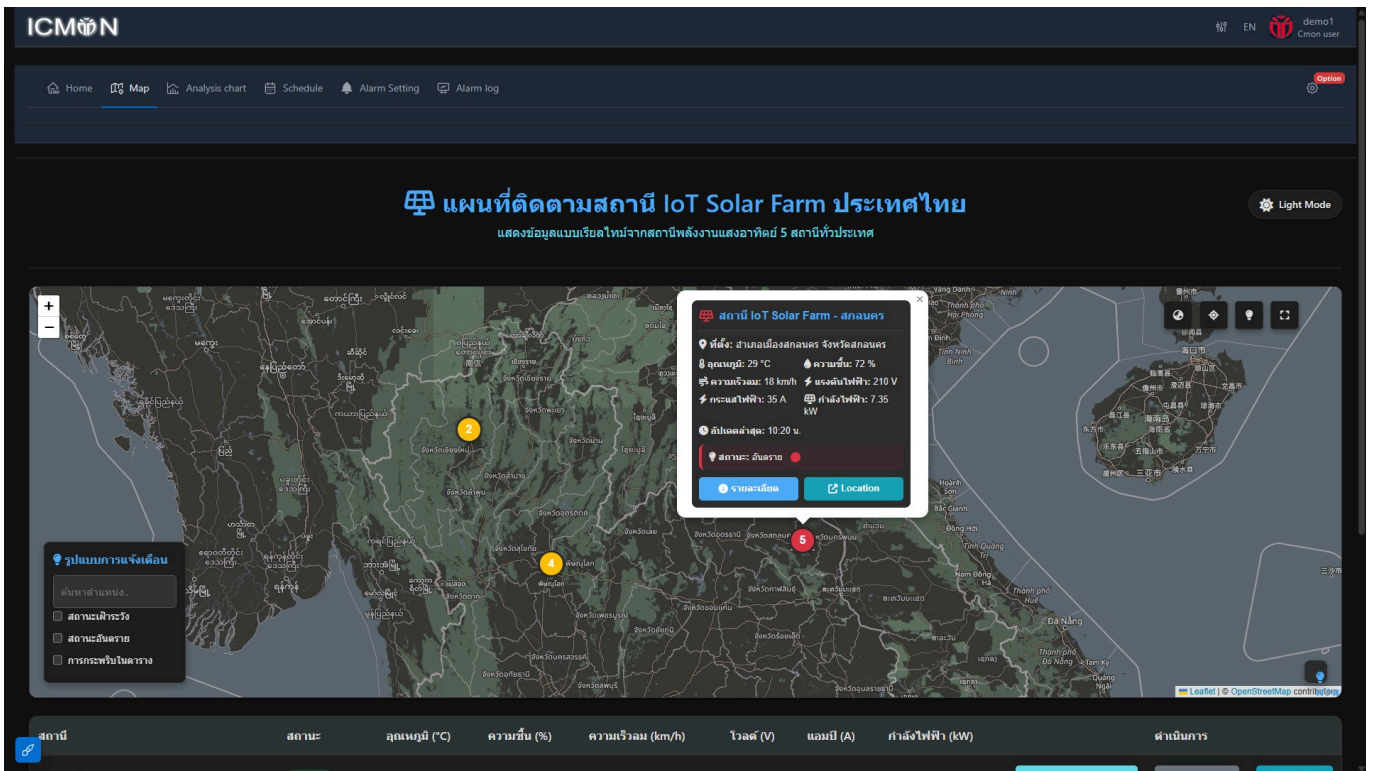
## 9. สรุปตารางเปรียบเทียบ Layer

Layer	โฟลเดอร์	รับผิดชอบ	ขึ้นอยู่กับอะไร
<b>Model</b>	<code>internal/models/</code>	กำหนด struct entity (GORM tags)	ไม่ขึ้นกับ layer อื่น
<b>Repository</b>	<code>internal/repository/</code>	อ่าน/เขียน DB, Redis (implement interface)	Model, DB driver
<b>Usecase</b>	<code>internal/usecase/</code>	Business logic, validation, hash, JWT, queue	Repository interface, pkg helpers
<b>Delivery (Handler)</b>	<code>internal/delivery/rest/handler/</code>	รับ HTTP request, เรียก usecase, ส่ง response	Usecase interface, DTO
<b>Delivery (Middleware)</b>	<code>internal/delivery/rest/middleware/</code>	ตรวจสอบ JWT, log, rate limit, CORS	pkg/logger, pkg/jwt
<b>Worker</b>	<code>internal/delivery/worker/</code>	ดึงงานจาก queue, ส่ง email	pkg/email, pkg/redis
<b>Shared Packages</b>	<code>internal/pkg/</code>	JWT, Redis client, email sender, hash, logger, validator, utils	third-party libs

หมายเหตุ: โครงสร้างนี้ ไม่เหมาะกับโปรเจกต์เล็ก ๆ ที่มีแค่ 2-3 endpoint เพราะ แต่เหมาะกับ ระบบที่มี business logic ซับซ้อน, ต้องทดสอบบ่อย, และมีโอกาสเปลี่ยนเทคโนโลยีในอนาคต

## ภาค 2







# การเพิ่มฟีเจอร์ใหม่ในโครงสร้าง icmongolang

## ตัวอย่าง: เพิ่ม Module IoT (ระบบส่งข้อมูลออนไลน์ + GPS Tracking + MQTT + Queue Processor)

โครงสร้างเดิมเป็น Clean Architecture ที่แยก Model → Repository → Usecase → Delivery อย่างชัดเจน การเพิ่ม module ใหม่จะทำแบบ **Modular** โดยไม่กระทบ module เดิม (auth, users)

### 1. ขั้นตอนการเพิ่มฟีเจอร์ใหม่ (ตามลำดับ)

#### ✓ ขั้นตอนที่ 1: วิเคราะห์และออกแบบ entity / data model

- ระบุ entity ที่เกี่ยวข้อง เช่น Device (IoT device), WaterOrder, GpsLocation, MqttMessage
- กำหนดความสัมพันธ์ (Device มีหลาย WaterOrder, Device มีหลาย GpsLocation)
- ออกแบบตารางใน PostgreSQL

ตัวอย่าง entity:

```
// internal/models/device.go
type Device struct {
 ID uint `gorm:"primaryKey"`
 SerialNo string `gorm:"uniqueIndex"`
 Name string
 UserID uint // เจ้าของ device (FK ไป users)
 Status string // active, inactive, maintenance
 CreatedAt time.Time
}

// internal/models/water_order.go
type WaterOrder struct {
 ID uint
 DeviceID uint
 Quantity int // ลิตร
 OrderTime time.Time
 Status string // pending, processing, completed, failed
 ProcessedAt *time.Time
}

// internal/models/gps_location.go
type GpsLocation struct {
 ID uint
 DeviceID uint
 Lat float64
```

```

 Lng float64
 Timestamp time.Time
}

```

## ✓ ขั้นตอนที่ 2: สร้าง Repository Interface + Implementation (PostgreSQL + Redis cache)

- สร้างไฟล์ `internal/repository/device_repo.go` (interface + GORM impl)
- สร้าง `internal/repository/water_order_repo.go`
- สร้าง `internal/repository/gps_repo.go` (อาจใช้ Redis สำหรับตำแหน่งล่าสุด)

ตัวอย่าง interface:

```

type DeviceRepository interface {
 Create(ctx context.Context, device *models.Device) error
 FindByID(ctx context.Context, id uint) (*models.Device, error)
 FindBySerial(ctx context.Context, serial string) (*models.Device, error)
 Update(ctx context.Context, device *models.Device) error
 Delete(ctx context.Context, id uint) error
 ListByUser(ctx context.Context, userID uint) ([]models.Device, error)
}

```

## ✓ ขั้นตอนที่ 3: สร้าง Usecase (Business Logic)

- สร้าง `internal/usecase/iot_usecase.go` (หรือแยกเป็น `device_usecase.go`, `order_usecase.go`)
- ใช้ repository หลายตัว, เรียก MQTT client, ส่งงานเข้า queue

ตัวอย่าง:

```

type IotUsecase interface {
 RegisterDevice(userID uint, serialNo, name string) (*models.Device, error)
 CreateWaterOrder(deviceID uint, quantity int) (*models.WaterOrder, error)
 PublishMqttCommand(deviceID uint, command string) error
 ProcessOrderQueue() // worker จะเรียก
 UpdateGpsLocation(deviceID uint, lat, lng float64) error
}

```

## ✓ ขั้นตอนที่ 4: สร้าง Handler (Delivery/REST)

- สร้าง `internal/delivery/rest/handler/device_handler.go`
- สร้าง `internal/delivery/rest/handler/order_handler.go`
- กำหนด DTO ใน `internal/delivery/rest/dto/iot_dto.go`

ตัวอย่าง handler:

```
func (h *DeviceHandler) CreateOrder(w http.ResponseWriter, r *http.Request) {
 // อ่าน deviceID จาก URL หรือ JWT claims
 // เรียก iotUsecase.CreateWaterOrder
 // response 201
}
```

## ✓ ขั้นตอนที่ 5: เพิ่ม Route ใน router.go

```
func (r *Router) SetupIotRoutes(deviceHandler *handler.DeviceHandler, authMiddleware func(http.Handler) http.Handler) {
 r.router.Route("/api/v1/devices", func(dev chi.Router) {
 dev.Use(authMiddleware)
 dev.Post("/", deviceHandler.RegisterDevice)
 dev.Get("/", deviceHandler.ListDevices)
 dev.Post("/{deviceID}/orders", deviceHandler.CreateOrder)
 dev.Post("/{deviceID}/command", deviceHandler.SendMqttCommand)
 })
}
```

## ✓ ขั้นตอนที่ 6: สร้าง MQTT Client (ใน internal/pkg/mqtt/ )

- ใช้ไลบรารี eclipse/paho.mqtt.golang
- สร้าง interface MqttPublisher , MqttSubscriber
- Subscribe topic เช่น device+/gps , device+/response

## ✓ ขั้นตอนที่ 7: สร้าง Queue Processor สำหรับ Water Order

- ใช้ Redis Stream หรือ Go channel (ตามโครงสร้างเดิมมี email queue อยู่แล้ว)
- สร้าง worker ใหม่: internal/delivery/worker/order\_worker.go
- worker จะดึง order จาก queue → ประมวลผล → ส่ง MQTT → อัปเดตสถานะ

## ✓ ขั้นตอนที่ 8: เพิ่ม Command ใน Cobra (ถ้าต้องการรัน worker แยก)

- เพิ่ม cmd/iotworker.go (หรือขยาย cmd/worker.go ให้รองรับหลาย queue)
- คำสั่ง: go run cmd/api/main.go iot-worker

## ✓ ขั้นตอนที่ 9: อัปเดต Docker Compose (ถ้าต้องการ MQTT broker)

- เพิ่ม service mosquitto (MQTT broker) ใน docker-compose.dev.yml
- เพิ่ม environment variables ใน .env.dev (MQTT\_BROKER\_URL, MQTT\_USERNAME, MQTT\_PASSWORD)

## ✓ ขั้นตอนที่ 10: ทดสอบและ Document API (Swagger)

- ใส่ annotation ใน handler เพื่อให้ swag สร้าง docs

- รัน `swag init` แล้วทดสอบที่ `/swagger/index.html`

หมายเหตุ: ขั้นตอนทั้งหมดสอดคล้องกับ Clean Architecture ของ `icmongolang` และสามารถนำไปปรับใช้กับพีเจอรอื่น ๆ ในรายการ (Smart Building, Smart Farm, Smart Healthcare, ฯลฯ) ได้โดยเปลี่ยน model และ usecase เท่านั้น ส่วน โครงสร้างหลัก (repository, delivery, pkg) จะเหมือนกัน

## การนำ Template ไปใช้

- ผู้เริ่มต้น ควรศึกษาเนื้อหาตามลำดับตั้งแต่ภาคที่ 1 ถึงภาคที่ 3
- <https://github.com/kongnakornna/icmongolang/blob/main/Book/BookGolang1.md>
- นักพัฒนาที่มีประสบการณ์ สามารถข้ามไปยังภาคที่ 7-9 เพื่อศึกษา Clean Architecture, DDD และการผสานระบบ ภายนอก
- [https://github.com/kongnakornna/icmongolang/tree/main/project\\_manamet/sulution](https://github.com/kongnakornna/icmongolang/tree/main/project_manamet/sulution)
- ทีมพัฒนา สามารถนำ Task List Template และ Checklist Template ไปปรับใช้ในการทำงาน
- [https://github.com/kongnakornna/icmongolang/tree/main/project\\_manamet/Template](https://github.com/kongnakornna/icmongolang/tree/main/project_manamet/Template)
- \*\*Design Thinking & Business Model Canvas (BMC) - Design & Implementation Package \*\*
- [https://github.com/kongnakornna/icmongolang/tree/main/project\\_manamet/DesignThinking](https://github.com/kongnakornna/icmongolang/tree/main/project_manamet/DesignThinking)

ผู้เขียน: คณกร จันทะคุณ

อีเมล: [kongnakornjantakun@gmail.com](mailto:kongnakornjantakun@gmail.com)



Line: kongnakornna

วันที่: เมษายน 2026

**\*\* ประวัติของผู้เขียน:\*\***

## เส้นทางนักพัฒนาซอฟต์แวร์: จากโค้ดบรรทัดแรกสู่ผู้นำทางเทคนิค

ทุกเส้นทางของนักพัฒนาซอฟต์แวร์มักเริ่มต้นจากความอยากรู้อยากเห็น และสำหรับผม เส้นทางนั้นเริ่มขึ้นเมื่อราว 15 ปี ก่อน ด้วยพื้นฐานด้านวิทยาการคอมพิวเตอร์จากมหาวิทยาลัยราชภัฏพิบูลสงคราม ก่อนจะขยับขยายขอบเขตความรู้ไปสู่ศาสตร์อีกด้านอย่างนิติศาสตร์ จากมหาวิทยาลัยสุโขทัยธรรมมาธิราช และประกาศนียบัตรทนายความจากสภานายความ ความรู้ด้านกฎหมายนี้เองที่กลายเป็นจุดแข็งสำคัญ ช่วยให้ผมมองเห็นภาพรวมของธุรกิจ เข้าใจข้อกำหนด และสื่อสารกับผู้ใช้งานได้อย่างมีประสิทธิภาพมากขึ้นในเวลาต่อมา

## จุดเริ่มต้นของนักปฏิบัติ

ปี 2549 ผมเริ่มต้นเส้นทางสายไอทีในฐานะ IT Support และ PHP Web Developer ที่บริษัท คอนโทรล ดาต้า (ประเทศไทย) จำกัด ช่วงเวลานั้นเป็นช่วงของการเรียนรู้พื้นฐานอย่างแท้จริง ทั้งการดูแล Data Center การให้บริการลูกค้า

และการพัฒนาเว็บไซต์ด้วย PHP ซึ่งเป็นรากฐานสำคัญที่ทำให้เข้าใจทั้งด้านเทคนิคและความต้องการของผู้ใช้ไปพร้อมกัน

## ก้าวสู่โลกของเนื้อหาคุณภาพ

ปี 2552 ผมได้ร่วมงานกับบริษัท สยามสปอร์ต ซินดิเคท จำกัด (มหาชน) ในตำแหน่ง Senior Developer นี่คือช่วงเวลาที่ได้พัฒนาทักษะการเขียน PHP และจัดการฐานข้อมูล MySQL, MSSQL ในการขับเคลื่อนเว็บไซต์ข่าวกีฬาชื่อดังอย่าง [siamสปอร์ต.co.th](http://siamспорт.co.th) ซึ่งต้องรองรับผู้เข้าชมจำนวนมากและต้องการประสิทธิภาพสูง การดูแลระบบ การทดสอบซอฟต์แวร์ และการฝึกอบรมผู้ใช้ในช่วงนี้ ทำให้เข้าใจความสำคัญของซอฟต์แวร์ที่เสถียรและใช้งานง่าย

## ขยายขอบเขตกับองค์กรใหญ่

ปี 2559 ถือเป็นอีกหนึ่งจุดเปลี่ยนสำคัญ เมื่อผมเข้าร่วมงานกับบริษัท ทู คอร์ปอเรชั่น จำกัด (มหาชน) ในตำแหน่ง Senior Full Stack Developer ที่นี่ผมได้ขยายขีดความสามารถจากการใช้ PHP ไปสู่ Node.js, TypeScript และการจัดการฐานข้อมูลที่หลากหลาย ทั้ง MySQL, MSSQL รวมถึงการใช้ ORM อย่าง TypeORM อย่างมืออาชีพ

ผลงานสำคัญในช่วงนี้คือการมีส่วนร่วมพัฒนาและปรับปรุงระบบการศึกษาระดับประเทศอย่าง [connexted.org](http://connexted.org) และ [trueplookpanya.com](http://trueplookpanya.com) ซึ่งเป็นแพลตฟอร์มที่เข้าถึงผู้ใช้จำนวนมาก การได้ร่วมกับทีมออกแบบกระบวนการทางธุรกิจ วางแผนพัฒนา ทำ UAT และให้การสนับสนุนผู้ใช้ เปิดมุมมองว่าการพัฒนาซอฟต์แวร์ไม่ได้จบที่การเขียนโค้ด แต่ต้องทำให้ผู้ใช้ใช้งานได้จริงและเกิดประโยชน์สูงสุด

## พลิกโฉมด้วยเทคโนโลยีใหม่

ปัจจุบัน ผมดำรงตำแหน่ง Technical Lead / Full Stack Software Engineer ที่บริษัท ซีทีซี โกลบอล (ประเทศไทย) จำกัด บทบาทนี้ท้าทายกว่าเดิมมาก เพราะนอกจากจะต้องพัฒนาระบบด้วย Node.js, TypeScript, Python แล้ว ยังต้องวางสถาปัตยกรรมซอฟต์แวร์ กำหนดมาตรฐานการเขียนโค้ด และดูแลทีมพัฒนา ทั้ง Developer, Application Support และ DevOps

สิ่งที่ทำให้ช่วงเวลานี้พิเศษคือการได้นำเทคโนโลยีสมัยใหม่มาประยุกต์ใช้อย่างจริงจัง:

- **ระบบอัตโนมัติ** ด้วย n8n และ UiPath RPA ช่วยลดงานซ้ำซ้อนและเพิ่มประสิทธิภาพการทำงาน
- **Generative AI** ผสาน OpenAI API เข้ากับแอปพลิเคชันเพื่อสร้างฟีเจอร์อัจฉริยะ
- **CI/CD Pipeline** ตั้งค่า Jenkins, GitHub Actions พร้อม Quality Gates (SonarQube, ESLint, Test Coverage) เพื่อให้การ deploy มั่นใจและปลอดภัย
- **ระบบ Monitoring** ด้วย Grafana และ ELK Stack เพื่อให้เห็นภาพรวมของระบบแบบ Real-time

## ความสำเร็จที่ภูมิใจ

ตลอดเส้นทางที่ผ่านมา มีผลงานหลายอย่างที่สร้างความภูมิใจ: – \*\*PoC <http://cmoniot.trueddns.com:52160> ที่นำ IoT มาติดตามอุณหภูมิ น้ำรั่ว และควัน พร้อม Dashboard แจ้งเตือนแบบ Real-time

- **การเป็นวิทยากรและที่ปรึกษา** ถ่ายทอดความรู้ด้าน DevOps, การเขียนโค้ดคุณภาพ, Design Thinking และ Business Model Canvas ให้ทีมพัฒนาทั้งภายในและภายนอกองค์กร
- **การลดข้อผิดพลาดในการ deploy** จากการนำ Branch Protection และ CI/CD มาบังคับใช้ เพิ่มความมั่นใจให้ทีม

## จุดแข็งที่สร้างความแตกต่าง

นอกเหนือจากทักษะทางเทคนิคที่หลากหลาย จุดแข็งที่ทำให้การทำงานประสบความสำเร็จคือ:

- **การสื่อสารที่มีประสิทธิภาพ** สามารถอธิบายเรื่องเทคนิคให้คนที่ไม่ใช่สายเทคนิคเข้าใจ และทำงานร่วมกับทีมธุรกิจได้อย่างราบรื่น
- **การแก้ปัญหาอย่างเป็นระบบ** วิเคราะห์หาสาเหตุที่แท้จริง และเสนอแนวทางที่เหมาะสม ไม่ใช่แค่แก้ปัญหาที่ปลายเหตุ
- **การเรียนรู้เร็วและปรับตัวเก่ง** ติดตามเทคโนโลยีใหม่อย่างสม่ำเสมอ และพร้อมนำมาประยุกต์ใช้เมื่อเห็นว่าจะเกิดประโยชน์
- **ภาวะผู้นำและการทำงานเป็นทีม** ให้คำปรึกษา สนับสนุนสมาชิก และสร้างบรรยากาศที่ทุกคนเติบโตไปด้วยกัน

ความมุ่งมั่นในการพัฒนาตนเองสะท้อนผ่านการเรียนคอร์สออนไลน์มากมาย ตั้งแต่ Data Science, Machine Learning, Docker & Kubernetes ไปจนถึง DevOps และการพัฒนา Full Stack ด้วย Angular & Nest JS รวมถึงการอบรม Business Model Canvas, Design Thinking, Agile & Scrum จาก True Digital Academy และ General Assembly

เส้นทางกว่าสิบห้าปีที่ผ่านมา สอนให้รู้ว่าการพัฒนาซอฟต์แวร์ที่ดีไม่ได้มีแค่ทักษะการเขียนโค้ด แต่ต้องเข้าใจธุรกิจ มีวิสัยทัศน์ในการออกแบบระบบ มีทักษะการนำทีม และที่สำคัญคือต้องมีใจที่อยากเรียนรู้และแบ่งปันเสมอ ผมเชื่อว่าทุกโครงการทุกบรรทัดโค้ด ที่เขียนขึ้น ล้วนเป็นส่วนหนึ่งในการขับเคลื่อนองค์กรและสร้างประสบการณ์ที่ดีให้ผู้ใช้ และนั่นคือความภาคภูมิใจของนักพัฒนาทุกคน

-----

```
go mod tidy go mod download go mod verify go run cmd/api/main.go migrate swag init -g cmd/api/main.go
mockery --all
go clean -cache
go clean -modcache go mod vendor go test ./...
air

go clean -cache
go clean -modcache
mockery --all
go test ./...
go build ./...
go test -v ./internal/modules/mqtt/delivery/http -run TestMQTTHandler_Publish go test -cover
./internal/modules/mqtt/...
go test ./internal/modules/mqtt/...
```

## 1. ล้าง cache ของ Go

```
go clean -cache go clean -modcache
```

## 2. จัดการ dependencies

---

go mod tidy go mod download go mod verify

## 3. Migrate ฐานข้อมูล (ใช้คำสั่งจาก cmd/api/main.go)

---

go run cmd/api/main.go migrate

## 4. สร้าง Swagger docs (ใช้ -g ชี้ไฟล์หลัก)

---

swag init -g cmd/api/main.go

## 5. Vendor dependencies (ถ้าต้องการ)

---

go mod vendor

## 6. รันทดสอบทุก package

---

go test ./...

## 7. รันด้วย Air (hot-reload) หรือใช้ go run

---

air

go clean -cache go clean -modcache go mod tidy go mod download go mod verify go run cmd/api/main.go migrate swag init -g cmd/api/main.go go mod vendor go test ./... air

- [http://localhost:5000/api/iot/monitordevicegroup?bucket=BAACTW01&location\\_id=&hardware\\_id=&lang=th&delcache=0](http://localhost:5000/api/iot/monitordevicegroup?bucket=BAACTW01&location_id=&hardware_id=&lang=th&delcache=0)
- db oracle Isnrctl start net start OracleServiceXE

net stop OracleServiceXE Isnrctl start net start OracleServiceXE Isnrctl status

make clean make tidy make download make verify make migrate make swag make vendor make test make run

go run main.go kafka

----

---

```
go mod tidy go mod download go mod verify go run cmd/api/main.go migrate swag init -g cmd/api/main.go
go clean -cache
go clean -modcache go mod vendor go test ./...
air ./air.cmd ./run.ps1 -All
```